

INFORMATIQUE • 1ÈRE TI

MODULE 1 : ALGORITHMIQUE

LEÇON N02

STRUCTURES DE DONNÉES ET SOUS-ALGORITHMES

DURÉE CONSEILLÉE : 10 H

DANS CETTE LEÇON

Cette leçon introduit les structures de données fondamentales (tableaux à une dimension et enregistrements) et les sous-algorithmes (fonctions et procédures). Elle apprend à organiser les données en mémoire et à découper un algorithme en blocs réutilisables, compétences centrales de l'épreuve d'informatique au Probatoire et au Baccalauréat TI.

- À la fin de cette leçon, je sais énumérer les principales structures de données et choisir celle adaptée à un problème
- À la fin de cette leçon, je sais déclarer, initialiser, remplir, parcourir, afficher et modifier un tableau à une dimension
- À la fin de cette leçon, je sais déclarer un enregistrement et accéder à ses champs en lecture et en écriture
- À la fin de cette leçon, je sais expliquer le rôle d'un sous-algorithme dans la résolution d'un problème
- À la fin de cette leçon, je sais déclarer et écrire une fonction simple et l'appeler dans un algorithme
- À la fin de cette leçon, je sais déclarer et écrire une procédure simple et l'appeler dans un algorithme



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Structures de données : panorama et tableaux
2. Les enregistrements
3. Sous-algorithmes : rôle et fonctions
4. Les procédures
5. Variables globales, variables locales, paramètres
6. Synthèse : structurer un algorithme complet
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais énumérer les principales structures de données et choisir celle adaptée à un problème
- À la fin de cette leçon, je sais déclarer, initialiser, remplir, parcourir, afficher et modifier un tableau à une dimension
- À la fin de cette leçon, je sais déclarer un enregistrement et accéder à ses champs en lecture et en écriture
- À la fin de cette leçon, je sais expliquer le rôle d'un sous-algorithme dans la résolution d'un problème
- À la fin de cette leçon, je sais déclarer et écrire une fonction simple et l'appeler dans un algorithme
- À la fin de cette leçon, je sais déclarer et écrire une procédure simple et l'appeler dans un algorithme
- À la fin de cette leçon, je sais distinguer une variable locale d'une variable globale et prévoir leur portée
- À la fin de cette leçon, je sais différencier paramètres formels et paramètres effectifs (arguments) lors d'un appel
- À la fin de cette leçon, je sais choisir entre fonction et procédure selon le résultat attendu

PRÉREQUIS

- Notion de variable, types simples (entier, réel, caractère, chaîne, booléen) et déclaration
- Structures de contrôle : si...alors...sinon, boucles pour et tant que
- Opérateurs arithmétiques et logiques, affectation
- Écriture d'un algorithme simple (en-tête, déclarations, corps) vue en début d'année
- Notion de fonction mathématique (ensemble de départ, image) vue en classe de Seconde

1

STRUCTURES DE DONNÉES : PANORAMA ET TABLEAUX

La CENADI (Centre National de Développement de l'Informatique) veut informatiser le suivi des élèves d'un lycée de Yaoundé. Le cahier des charges est clair : pour chaque élève, on veut mémoriser son nom, sa classe, ses notes de trois trimestres et sa moyenne. Le proviseur pose alors trois questions très concrètes à l'informaticien :

- Comment ranger en mémoire les notes des 60 élèves d'une classe sans créer 60 variables différentes ?
- Comment regrouper sous un seul nom toutes les informations d'un élève (nom, classe, notes...)?
- Comment éviter de réécrire dix fois le même calcul de moyenne dans un long algorithme ?

Cette leçon donne les outils pour répondre à ces trois questions : les **tableaux**, les **enregistrements** et les **sous-algorithmes**. Dans cette première section, nous nous concentrons sur le panorama des structures de données et sur la plus utilisée d'entre elles : le tableau à une dimension.

1.1 De la variable simple à la structure de données

Le problème : la limite de la variable simple.

Tu sais déjà déclarer et utiliser une variable simple :

Variable Note1 : Reel

Une variable simple ne peut contenir qu'une seule valeur à la fois. Pour mémoriser les notes de 60 élèves, il faudrait donc écrire :

Variables Note1, Note2, Note3, ..., Note60 : Reel

C'est une catastrophe pratique : 60 lignes de déclaration, 60 instructions `Lire` pour la saisie, et pour calculer la moyenne de la classe, une addition de 60 termes écrite à la main ! Pire encore : aucune boucle `Pour` ne peut parcourir 60 variables aux noms différents, car on ne peut pas « fabriquer » un nom de variable pendant l'exécution. Le programme devient impossible à écrire, à lire et à maintenir.

Il faut donc un moyen de stocker plusieurs valeurs sous un seul nom, tout en pouvant désigner chacune d'elles individuellement. C'est exactement le rôle d'une structure de données.

DÉFINITION Structure de données

Une **structure de données** est une manière organisée de stocker et d'accéder à un ensemble de valeurs dans la mémoire de l'ordinateur. Elle définit à la fois l'organisation interne des données (comment elles sont rangées) et les opérations que l'on peut effectuer sur elles (comment on les lit, les écrit, les parcourt).

Panorama des structures de données usuelles.

En algorithmique, on distingue classiquement :

Structure	Description
Types simples	Entier, Réel, Caractère, Booléen, Chaîne : une seule valeur par variable.
Tableau	Ensemble de cases de même type, repérées par un indice.
Enregistrement	Regroupement de champs de types éventuellement différents sous un seul nom.
Fichier	Stockage permanent de données sur un support (disque). (Complément, niveau Bac.)
Liste, pile, file	Structures dynamiques chaînées. (Complément, niveau Bac.)

Au programme de Première TI, nous étudions en détail les **tableaux** (cette section) et les **enregistrements** (section suivante). Les fichiers, listes, piles et files sont mentionnés ici à titre de culture générale : ils seront approfondis en Terminale.

1.2 Le tableau à une dimension

Intuition.

Imagine une rangée de 60 casiers numérotés de 1 à 60 dans la salle des professeurs. Tous les casiers ont la même forme et contiennent le même type d'objet (des copies). Pour prendre les copies du 5^e élève, tu ne cherches pas un « casier Dupont » : tu vas directement au casier numéro 5. Le tableau fonctionne exactement ainsi : un seul nom (le nom du tableau), et un numéro (l'indice) pour désigner chaque case.

DÉFINITION Tableau à une dimension

Un **tableau à une dimension** est une structure de données constituée d'un ensemble fini de cases mémoire (appelées éléments), toutes de même type, et repérées par un **indice**. Un tableau est caractérisé par :

- son nom (commun à toutes les cases) ;
- sa taille : le nombre de cases, fixé à la déclaration ;
- le type des éléments : le type commun des valeurs stockées ;
- ses bornes : la borne inférieure (premier indice) et la borne supérieure (dernier indice).

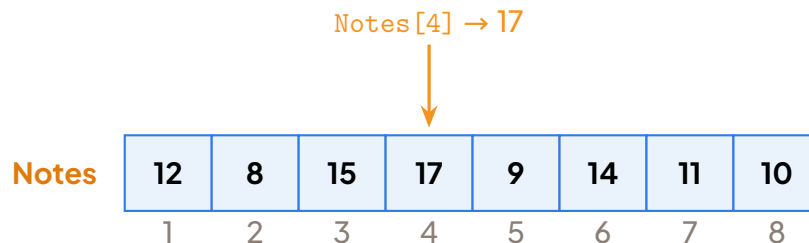
PROPRIÉTÉ Syntaxe de déclaration d'un tableau

En écriture algorithmique, la déclaration d'un tableau se fait dans la zone des déclarations selon la syntaxe :

`Tableau NomTableau[borneInf..borneSup] : Type`

Exemple : `Tableau Notes[1..60] : Reel` déclare un tableau nommé `Notes`, de 60 cases

numérotées de 1 à 60, chaque case pouvant contenir un nombre réel. La taille du tableau est : $\text{borneSup} - \text{borneInf} + 1$, ici $60 - 1 + 1 = 60$ cases.



Le tableau `Notes[1..8]` en mémoire : 8 cases de même type, repérées par leur indice. L'accès `Notes[4]` désigne directement la 4^e case, qui contient la valeur 17.

Accès à un élément par son indice.

Pour lire ou modifier le contenu d'une case, on écrit le nom du tableau suivi de l'indice entre crochets :

```
Notes[5] <- 14,5    // ecriture : la case 5 recoit la valeur 14,5
Ecrire(Notes[5])     // lecture : affiche le contenu de la case 5
Somme <- Notes[1] + Notes[2] // utilisation dans un calcul
```

L'indice peut être un nombre, une variable ou même une expression : `Notes[i]`, `Notes[i+1]` sont parfaitement valides, à condition que la valeur de l'indice reste entre les bornes déclarées. C'est cette possibilité d'utiliser une variable comme indice qui rend les tableaux si puissants : une boucle peut ainsi parcourir toutes les cases une par une.

ATTENTION Dépassement de borne : l'erreur classique

Si le tableau est déclaré `Tableau Notes[1..60] : Reel`, alors les seuls indices valides sont 1, 2, ..., 60. Écrire `Notes[0]` ou `Notes[61]` est une erreur de **dépassement de borne** (ou dépassement d'indice) : en algorithmique c'est une faute logique, et dans un vrai langage cela provoque un plantage du programme ou, pire, la modification silencieuse d'une autre zone de la mémoire. Vérifie toujours que tes indices restent dans l'intervalle `[borneInf ; borneSup]`, en particulier dans les boucles : pour un tableau `[1..60]`, la boucle doit aller de 1 à 60, jamais de 0 à 60 ni de 1 à 61.

1.3 Les cinq opérations de base sur un tableau

Tout traitement de tableau repose sur cinq opérations fondamentales qu'il faut savoir écrire les yeux fermés.

MÉTHODE Les 5 opérations de base sur un tableau

1. Déclarer : choisir le nom, les bornes et le type des éléments.
`Tableau Notes[1..60] : Reel`
2. Initialiser : donner une valeur de départ à chaque case (souvent 0) à l'aide d'une boucle `Pour`.
3. Remplir : saisir les valeurs au clavier, case par case, avec une boucle `Pour` et l'instruction `Lire`.
4. Parcourir / afficher : lire successivement toutes les cases avec une boucle `Pour` et l'instruction `Ecrire`.
5. Modifier : changer le contenu d'une case précise en l'affectant directement, par exemple `Notes[3] <- 12`.

Initialisation.

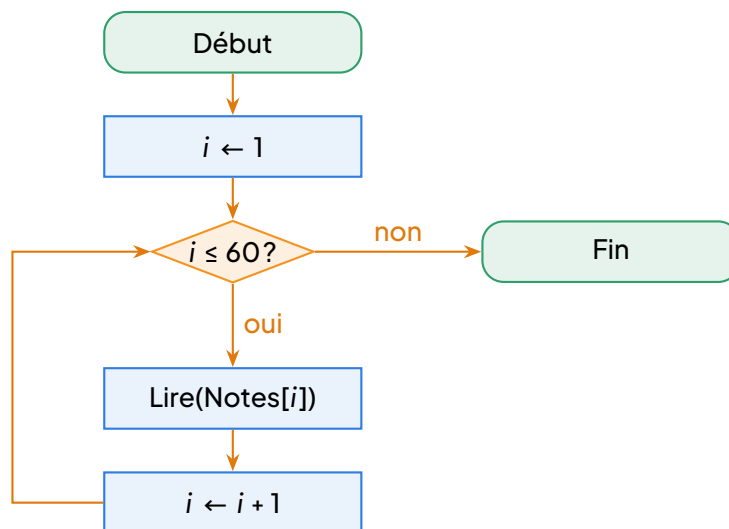
À la déclaration, le contenu des cases est inconnu (des « restes » de la mémoire). Il est donc prudent de tout mettre à zéro avant utilisation :

```
Pour i <- 1 a 60 Faire  
    Notes[i] <- 0  
FinPour
```

Remplissage par saisie au clavier.

```
Pour i <- 1 a 60 Faire  
    Ecrire("Entrez la note de l'élève numero ", i, " : ")  
    Lire(Notes[i])  
FinPour
```

À chaque tour de boucle, la variable `i` prend la valeur suivante (1, puis 2, puis 3...) et l'instruction `Lire(Notes[i])` range la valeur tapée dans la case correspondante. Une seule boucle remplace 60 instructions `Lire` : c'est toute la puissance du couple tableau + boucle.



Organigramme de la boucle de remplissage : tant que $i \leq 60$, on saisit la case d'indice i , puis on incrémente i .

Parcours et affichage.

```

Pour i ← 1 à 60 Faire
    Ecrire("Eleve", i, " : ", Notes[i])
FinPour
  
```

Modification d'un élément.

Pour corriger la note du 3^e élève, on affecte directement la case visée, sans toucher aux autres :

```
Notes[3] ← 12,5
```

1.4 Exemple chiffré complet : la moyenne de la classe

Mettons tout cela en musique avec un algorithme complet, du type de ceux que la CENADI pourrait intégrer dans son logiciel de suivi scolaire.

EXEMPLE Saisie, affichage et moyenne des notes de 10 élèves

Algorithme MoyenneClasse

Variables

i : Entier

Somme, Moyenne : Reel

Tableau Notes[1..10] : Reel

Debut

// 1. Remplissage du tableau

Pour $i \leftarrow 1$ à 10 Faire

Ecrire("Note de l'eleve", i , " : ")

```
    Lire(Notes[i])
  FinPour

  // 2. Affichage de toutes les notes
  Ecrire("--- Recapitulatif des notes ---")
  Pour i <- 1 à 10 Faire
    Ecrire("Eleve ", i, " : ", Notes[i], " / 20")
  FinPour

  // 3. Calcul de la moyenne de la classe
  Somme <- 0
  Pour i <- 1 à 10 Faire
    Somme <- Somme + Notes[i]
  FinPour
  Moyenne <- Somme / 10
  Ecrire("Moyenne de la classe : ", Moyenne, " / 20")
Fin
```

Remarque la structure générale : une boucle par traitement. On ne cherche jamais à tout faire dans une seule boucle géante ; chaque boucle a un rôle précis (remplir, afficher, accumuler). Cette clarté est une qualité essentielle d'un bon algorithme.

EXERCICE RÉSOLU 1 Calculer la moyenne à la main

Énoncé. On exécute l'algorithme `MoyenneClasse` avec les 10 notes suivantes : 12 ; 8 ; 15 ; 17 ; 9 ; 14 ; 11 ; 10 ; 13 ; 16. Quelle valeur est affichée à la fin ?

SOLUTION

Solution détaillée. La troisième boucle accumule les notes dans la variable `Somme` :

$$\begin{aligned}\text{Somme} &= 12 + 8 + 15 + 17 + 9 + 14 + 11 + 10 + 13 + 16 \\ &= 125\end{aligned}$$

Puis l'algorithme calcule :

$$\text{Moyenne} = \frac{\text{Somme}}{10} = \frac{125}{10} = 12,5$$

L'algorithme affiche donc : « Moyenne de la classe : 12,5 / 20 ». On remarque au passage que la note de l'élève 4 (17) est bien celle de la case `Notes[4]`, conformément au schéma du tableau vu plus haut.

ATTENTION Pièges fréquents avec les tableaux

- Oublier d'initialiser `Somme` à 0 avant la boucle d'accumulation : la moyenne obtenue est alors fausse et imprévisible.
- Confondre l'indice et la valeur : `Notes[i]` est la valeur contenue dans la case ; `i` n'est que le numéro de la case. Écrire `Somme <- Somme + i` additionne les numéros, pas les notes!
- Dépasser les bornes : boucler de 0 à 10 sur un tableau déclaré `[1..10]` provoque un accès interdit à `Notes[0]`.
- Déclarer une taille trop petite : si la classe compte 60 élèves, un tableau `[1..10]` ne suffira jamais. La taille se choisit dès la déclaration, en fonction du besoin.

LESAVAIS-TU ? Les indices commencent-ils à 0 ou à 1?

En écriture algorithmique au lycée, on choisit librement les bornes, et l'habitude francophone est de commencer à 1 : `Notes[1..60]`. Mais dans beaucoup de langages réels que tu rencontreras — C, Java, JavaScript, Python —, le premier élément d'un tableau porte toujours l'indice 0 : un tableau de 60 cases `y` est numéroté de 0 à 59, et le dernier indice valide est donc la taille moins un. Cette différence est une source d'erreurs célèbre chez les programmeurs débutants (le fameux `off-by-one error`). D'où l'habitude professionnelle à prendre dès maintenant : toujours vérifier les bornes déclarées avant d'écrire une boucle de parcours.

À RETENIR L'essentiel de la section

- Une variable simple ne contient qu'une valeur ; une **structure de données** organise le stockage et l'accès à un ensemble de valeurs.
- Les structures usuelles sont : les types simples, les **tableaux** et les **enregistrements** (les fichiers et listes seront vus plus tard).
- Un tableau à une dimension est un ensemble de cases de même type, repérées par un indice, déclaré par `Tableau Nom[borneInf..borneSup] : Type`.
- On accède à une case par son indice : `Notes[i]`, en lecture comme en écriture.
- Les cinq opérations de base : déclarer, initialiser, remplir, parcourir/afficher, modifier — chacune s'appuie sur une boucle `Pour`.
- Tout accès hors des bornes déclarées est une erreur grave : vérifie toujours tes indices.

EXERCICE 1 À toi de jouer

1. Déclare un tableau `Moyennes` capable de contenir les moyennes annuelles des 60 élèves d'une classe de Première TI.
2. Écris l'algorithme qui remplit ce tableau par saisie, puis affiche uniquement les moyennes supérieures ou égales à 10 (les admis).
3. Modifie l'algorithme `MoyenneClasse` pour qu'il affiche en plus la note la plus élevée de la classe. (Indice : parcourir le tableau en mémorisant la plus grande valeur rencontrée.)

CORRIGÉ Exercice 1

1. Déclaration : Tableau `Moyennes[1..60]` : Reel
2. Algorithme de saisie et d'affichage des admis :

```

Algorithme Admis
Variables
  i : Entier
  Tableau Moyennes[1..60] : Reel
Debut
  Pour i <- 1 a 60 Faire
    Ecrire("Moyenne de l'eleve ", i, " : ")
    Lire(Moyennes[i])
  FinPour
  Ecrire("--- Eleves admis ---")
  Pour i <- 1 a 60 Faire
    Si Moyennes[i] >= 10 Alors
      Ecrire("Eleve ", i, " : ", Moyennes[i])
    FinSi
  FinPour
Fin

```

3. On ajoute une variable `Max` initialisée à la première case, puis on la compare à chaque élément :

```

Max <- Notes[1]
Pour i <- 2 a 10 Faire
  Si Notes[i] > Max Alors
    Max <- Notes[i]
  FinSi
FinPour
Ecrire("Meilleure note : ", Max, " / 20")

```

Attention à deux détails : on initialise `Max` avec `Notes[1]` (et non 0, ce qui serait faux si toutes les notes étaient négatives), et la boucle démarre à 2 puisque la case 1 est déjà prise en compte.

2 LES ENREGISTREMENTS

2.1 Pourquoi le tableau ne suffit pas

Dans la section précédente, nous avons découvert le **tableau** : une structure de données qui regroupe plusieurs valeurs de même type sous un seul nom, accessibles par un indice. Un tableau `Notes[1..30] : Réel` contient 30 réels, un tableau `Noms[1..30] : Chaîne` contient 30 chaînes. Jusque-là, tout va bien.

Mais observons une situation très concrète, celle du proviseur d'un lycée de Yaoundé qui veut gérer les élèves d'une classe de Première TI. Pour décrire un élève, il faut :

- son **nom** : une chaîne de caractères (« Ngo Bell »);
- son **âge** : un entier (17);
- sa **moyenne** : un réel (12,75).

Ces trois informations sont de types différents. Si l'on s'en tient aux tableaux, il faut en créer trois :

```
Var Noms[1..60] : Chaîne  
Var Ages[1..60] : Entier  
Var Moyens[1..60] : Réel
```

L'élève numéro i est alors dispersé : son nom est dans `Noms[i]`, son âge dans `Ages[i]`, sa moyenne dans `Moyens[i]`. Cette solution fonctionne, mais elle est fragile : rien ne garantit que les trois tableaux restent synchronisés (un tri sur les noms oubliera les âges), et l'algorithme devient vite illisible. Le besoin est clair : il nous faut une structure qui regroupe, sous un seul nom, des informations de types différents décrivant un même objet. Cette structure existe : c'est l'**enregistrement**.

2.2 Définition et syntaxe de déclaration

DÉFINITION Enregistrement et champ

Un **enregistrement** est une structure de données qui regroupe, sous un seul nom, plusieurs variables appelées **champs**, ces champs pouvant être de types différents. Chaque champ possède un nom et un type. On accède à un champ d'une variable enregistrement par la **notation pointée** :

`variable.champ`

Par exemple, si `E` est un enregistrement possédant un champ `Moy`, alors `E.Moy` désigne la valeur de ce champ.

Contrairement aux types simples (Entier, Réel, Chaîne, Booléen), un enregistrement est un type construit par le programmeur : il faut d'abord **déclarer le type** (le « moule »), puis **déclarer des variables** de ce type (les « objets » fabriqués avec ce moule).

PROPRIÉTÉ Syntaxe de déclaration d'un type enregistrement

La déclaration d'un type enregistrement se fait dans la partie déclarative de l'algorithme, avant les variables :

```
Type NomDuType = Enregistrement
  champ1 : Type1
  champ2 : Type2
  ...
  champN : TypeN
Fin
```

Ensuite, on déclare une variable de ce type comme n'importe quelle variable :

```
Var E : NomDuType
```

Pour notre élève, le moule s'écrit :

```
Type Eleve = Enregistrement
  Nom : Chaîne
  Age : Entier
  Moy : Reel
Fin
Var E : Eleve
```

ATTENTION Type ou variable ? Ne pas confondre !

L'erreur la plus fréquente chez les débutants est de confondre le type `Eleve` (le moule, une définition abstraite) et la variable `E` (un objet concret qui contient des valeurs). On accède aux champs via la variable, jamais via le type :

- **Correct** : `E.Moy ← 12,75`
- **Faux** : `Eleve.Moy ← 12,75` — `Eleve` est un type, il ne contient aucune valeur !

De même, on ne peut pas écrire `Ecrire(E)` pour afficher tout l'enregistrement d'un coup : il faut afficher les champs un par un.

2.3 Manipuler un enregistrement : initialiser, accéder, afficher, modifier

MÉTHODE Déclarer et utiliser un type enregistrement en 5 étapes

1. Déclarer le type : écrire le bloc `Type ... = Enregistrement ... Fin` en listant chaque

champ avec son type.

2. **Déclarer la variable** : `Var E : Eleve.`
3. **Initialiser champ par champ** : par affectation (`E.Nom ← "Ngo Bell"`) ou par saisie (`Lire(E.Age)`).
4. **Accéder / afficher** : utiliser la notation pointée dans les expressions et les affichages (`Ecrire(E.Moy)`).
5. **Modifier un champ** : une simple affectation sur le champ concerné (`E.Age ← E.Age + 1`); les autres champs ne sont pas touchés.

Voici un algorithme complet qui met en musique ces cinq étapes :

```
Algorithme FicheEleve
Type Eleve = Enregistrement
  Nom : Chaîne
  Age : Entier
  Moy : Reel
Fin
Var E : Eleve
Debut
  // Initialisation par saisie, champ par champ
  Ecrire("Nom de l'eleve : ")
  Lire(E.Nom)
  Ecrire("Age de l'eleve : ")
  Lire(E.Age)
  Ecrire("Moyenne de l'eleve : ")
  Lire(E.Moy)

  // Affichage des champs
  Ecrire("Eleve : ", E.Nom, " – Age : ", E.Age)
  Ecrire("Moyenne : ", E.Moy)

  // Modification d'un seul champ : bon anniversaire !
  E.Age ← E.Age + 1
  Ecrire("Après son anniversaire : ", E.Age, " ans")
Fin
```

EXEMPLE Trace d'exécution

Si l'utilisateur saisit « Ngo Bell », 17 et 12,75, l'algorithme affiche :

	Eleve : Ngo Bell – Age : 17
	Moyenne : 12,75
	Après son anniversaire : 18 ans

Remarque : la modification de `E.Age` n'a altéré ni `E.Nom` ni `E.Moy`. Chaque champ vit sa propre vie à l'intérieur de l'enregistrement.

La figure suivante représente l'enregistrement `E` en mémoire : un seul bloc, trois zones.



Un enregistrement `E` de type `Eleve` : trois champs de types différents regroupés sous un seul nom. La notation pointée `E.Moy` cible le champ `Moy`.

À RETENIR

L'essentiel sur les enregistrements

- Un enregistrement regroupe des champs de types différents sous un seul nom ; un tableau regroupe des éléments de même type.
- On déclare d'abord le type (`Type Eleve = Enregistrement ... Fin`), puis la variable (`Var E : Eleve`).
- Toute manipulation (saisie, affectation, affichage, calcul) se fait champ par champ avec la notation pointée `variable.champ`.

2.4 Complément utile au Bac : le tableau d'enregistrements

LES AVAIS-TU ?

De l'enregistrement à la base de données

L'enregistrement est l'ancêtre direct de la ligne d'une table de base de données : chaque champ correspond à une colonne. Quand tu créeras en fin d'année une table `ELEVE` sous MS Access (module Systèmes d'information), chaque ligne de la table sera exactement un enregistrement : un nom, un âge, une moyenne. Les concepteurs des langages COBOL (1959) puis Pascal (1970) ont inventé cette structure précisément pour modéliser les fiches administratives — les mêmes fiches que remplissent encore aujourd'hui les secrétariats de nos lycées !

Revenons à notre classe de 60 élèves. Nous savons maintenant décrire un élève ; comment en décrire soixante ? En combinant les deux structures étudiées : un **tableau dont chaque case est**

un enregistrement. C'est le meilleur des deux mondes : le tableau donne la répétition, l'enregistrement donne la richesse des types.

PROPRIÉTÉ Déclaration et accès à un tableau d'enregistrements

On déclare un tableau dont le type des éléments est le type enregistrement :

```
Var Classe[1..60] : Eleve
```

On accède alors à un champ en combinant l'indice et la notation pointée :

`Classe[i].Moy` désigne la moyenne de l'élève numéro *i*.

L'indice se met sur la variable tableau, le point sur l'enregistrement obtenu : `Classe[2]` est un enregistrement, donc `Classe[2].Nom` est son champ `Nom`.

Classe[75]

Nom	Age	Moy
75	75	75

Classe[50]

Nom	Age	Moy
-----	-----	-----

ATTENTION Où mettre le point, où mettre l'indice ?

Deux pièges classiques avec les tableaux d'enregistrements :

- Écrire `Classe.Nom[2]` est faux : `Classe` est un tableau, on doit d'abord choisir la case (`Classe[2]`), puis le champ (`.Nom`).
- Oublier que chaque case doit être initialisée champ par champ : `Lire(Classe[i])` ne fonctionne pas ; il faut `Lire(Classe[i].Nom)`, `Lire(Classe[i].Age)`, etc.

Retiens l'ordre de lecture : `Classe[2].Nom` se lit « la case 2 du tableau `Classe`, puis son champ `Nom` ».

2.5 Exemple résolu : le plus jeune de trois élèves

EXERCICE RÉSOLU 2 Saisie de 3 élèves et recherche du plus jeune

Énoncé. Écrire un algorithme qui saisit les informations (nom, âge, moyenne) de 3 élèves dans un tableau d'enregistrements, puis affiche le nom de l'élève le plus jeune.

SOLUTION

Solution détaillée. La démarche suit trois phases : déclaration du type et du tableau, saisie par une boucle `Pour`, puis recherche du minimum sur le champ `Age` en mémorisant l'indice du plus jeune (et non son âge, car c'est son nom qu'il faut afficher).

```

Algorithme PlusJeune
Type Eleve = Enregistrement
    Nom : Chaîne
    Age : Entier
    Moy : Reel
Fin
Var Classe[1..3] : Eleve
Var i, indMin : Entier
Debut
    // Phase 1 : saisie des 3 enregistrements
    Pour i De 1 A 3 Faire
        Ecrire("Eleve n", i, " - Nom : ")
        Lire(Classe[i].Nom)
        Ecrire("Age : ")
        Lire(Classe[i].Age)
        Ecrire("Moyenne : ")
        Lire(Classe[i].Moy)
    FinPour

    // Phase 2 : recherche de l'indice du plus jeune
    indMin <- 1

```

```

    Pour i De 2 A 3 Faire
        Si Classe[i].Age < Classe[indMin].Age Alors
            indMin ← i
        FinSi
    FinPour

    // Phase 3 : affichage du resultat
    Ecrire("Le plus jeune est : ", Classe[indMin].Nom)
Fin

```

Trace avec les données de la figure (Ngo Bell, 17 ; Etoa Jean, 16 ; Mbida Rose, 18) :

- Initialisation : $\text{indMin} \leftarrow 1$ (âge de référence : $\text{Classe}[1].\text{Age} = 17$).
- $i = 2$: $\text{Classe}[2].\text{Age} = 16 < 17$, donc $\text{indMin} \leftarrow 2$.
- $i = 3$: $\text{Classe}[3].\text{Age} = 18$, non inférieur à 16, indMin reste 2.

L'algorithme affiche : Le plus jeune est : Etoa Jean. Remarque l'élégance de la solution : grâce au tableau d'enregistrements, le nom, l'âge et la moyenne de chaque élève restent solidaires ; on n'a jamais à jongler avec trois tableaux parallèles.

EXERCICE 2 Mérite de la classe

En t'inspirant de l'exemple résolu, écris un algorithme qui saisit les informations de 5 élèves dans un tableau d'enregistrements, puis affiche le nom et la moyenne de l'élève ayant la meilleure moyenne. Tu réutiliseras le type `Eleve` et la technique de mémorisation d'un indice.

CORRIGÉ Exercice : Mérite de la classe

Il suffit d'adapter la recherche : on compare le champ `Moy` et l'on cherche un maximum.

```

Algorithme MeilleurEleve
Type Eleve = Enregistrement
    Nom : Chaîne
    Age : Entier
    Moy : Reel
Fin

Var Classe[1..5] : Eleve
Var i, indMax : Entier
Debut
    Pour i De 1 A 5 Faire
        Ecrire("Eleve n", i, " - Nom : ")
        Lire(Classe[i].Nom)
        Ecrire("Age : ")
    Fin

```

```

Lire(Classe[i].Age)
Ecrire("Moyenne : ")
Lire(Classe[i].Moy)
FinPour

indMax <- 1
Pour i De 2 A 5 Faire
    Si Classe[i].Moy > Classe[indMax].Moy Alors
        indMax <- i
    FinSi
FinPour

Ecrire("Major de la classe : ", Classe[indMax].Nom)
Ecrire("Moyenne : ", Classe[indMax].Moy)
Fin

```

Deux points de vigilance : la comparaison porte sur `Moy` avec le symbole `>` (maximum, pas minimum), et l’affichage final utilise deux champs du même enregistrement `Classe[indMax]`, ce qui montre bien que toutes les informations de l’élève restent regroupées.

Tu maîtrises désormais les deux structures de données fondamentales du programme : le tableau (même type, accès par indice) et l’enregistrement (types variés, accès par champ). Il reste un problème de taille : nos algorithmes s’allongent et mélangent tout — saisie, calculs, affichage. La prochaine section introduit les sous-algorithmes (fonctions et procédures), qui permettent de découper un gros problème en petits morceaux réutilisables.

3 SOUS-ALGORITHMES : RÔLE ET FONCTIONS

Dans la section précédente, nous avons appris à organiser les données grâce aux tableaux et aux enregistrements. Mais un algorithme, ce n’est pas seulement des données : c’est aussi des traitements. Et lorsque les traitements deviennent longs et répétitifs, un nouveau problème apparaît. C’est précisément ce problème que les **sous-algorithmes** viennent résoudre. Dans cette section, nous découvrons leur rôle, puis nous étudions en détail la première famille de sous-algorithmes : les **fonctions**.

3.1 Le constat : un long algorithme répète les mêmes traitements

Imagine que tu écrives un algorithme de gestion des notes pour le conseil de classe de ta Première TI. À plusieurs endroits, tu dois calculer une moyenne de deux notes : moyenne des séquences 1 et 2, moyenne des séquences 3 et 4, moyenne des séquences 5 et 6. Sans outil particulier, tu écrirais trois fois les mêmes instructions :

```

M1 <-- (S1 + S2) / 2
M2 <-- (S3 + S4) / 2
M3 <-- (S5 + S6) / 2

```

Trois fois le même calcul, avec seulement les données qui changent. Et si le directeur décide demain que la moyenne se calcule autrement (par exemple avec des coefficients)? Il faudra corriger l'algorithme à trois endroits, sans en oublier un seul, sous peine d'obtenir des résultats incohérents. C'est exactement le genre de situation qui rend un programme long, fragile et difficile à maintenir.

L'idée fondamentale est alors la suivante : **découper le problème en blocs nommés et réutilisables**. On écrit le calcul une seule fois, on lui donne un nom (par exemple *Moyenne*), et on le demande (on dit : on l'appelle) chaque fois qu'on en a besoin, en lui transmettant les données du moment. C'est le principe du **sous-algorithme**.

DÉFINITION Sous-algorithme

Un **sous-algorithme** est un bloc d'instructions nommé, écrit à part de l'algorithme principal, qui réalise une tâche précise et qui peut être appelé (exécuté) depuis l'algorithme principal, autant de fois que nécessaire. On distingue deux familles de sous-algorithmes : les **fonctions** et les **procédures**.

3.2 Rôles des sous-algorithmes

Pourquoi découper un algorithme en sous-algorithmes? Quatre raisons essentielles, à connaître pour le Probatoire :

- **Éviter les répétitions.** Un traitement utilisé plusieurs fois n'est écrit qu'une seule fois. L'algorithme est plus court et toute correction se fait en un seul endroit.
- **Rendre l'algorithme lisible.** L'algorithme principal devient un enchaînement d'appels aux noms parlants (*Moyenne*, *Max2*, *AfficherBulletin*) : on comprend la logique générale sans se noyer dans les détails.
- **Faciliter la correction et les tests.** Chaque sous-algorithme peut être vérifié isolément. Si la moyenne est fausse, on sait exactement quel bloc inspecter.
- **Permettre le travail en équipe.** Dans un projet réel (par exemple le développement d'une application de gestion de stock pour une boutique de Douala), chaque membre de l'équipe peut écrire ses propres sous-algorithmes, à condition que chacun respecte le nom, les paramètres et le résultat annoncés.

LESAVAIS-TU ? Diviser pour régner

Cette stratégie de découpage porte un nom célèbre en informatique : diviser pour régner (divide and conquer). Un problème trop gros est découpé en sous-problèmes plus simples, que l'on résout séparément avant de combiner les résultats. C'est la même philosophie que celle d'un chantier : on ne construit pas une maison d'un seul bloc, on confie la maçonnerie, la plomberie et l'électricité à des corps de métier spécialisés.

3.3 Les deux familles : fonctions et procédures

Il existe deux sortes de sous-algorithmes :

- la **fonction** : elle reçoit des données (les paramètres), effectue un calcul et **retourne un seul résultat** à celui qui l'a appelée ;
- la **procédure** : elle reçoit éventuellement des données et effectue un traitement (affichage, modification de variables, etc.) **sans retourner de résultat**.

La différence essentielle est donc là : **une fonction retourne une valeur, une procédure non**. Nous étudions les fonctions maintenant ; les procédures feront l'objet de la section suivante.

3.4 Définition et syntaxe d'une fonction

DÉFINITION Fonction

Une **fonction** est un sous-algorithme qui reçoit des paramètres (des données en entrée), effectue un traitement et retourne un seul résultat typé (entier, réel, caractère, booléen, chaîne) à l'algorithme appelant, grâce à l'instruction `Retourner`.

L'analogie avec les mathématiques est immédiate. La fonction $f(x) = 2x + 1$ reçoit un nombre x et produit un seul nombre $f(x)$. En algorithmique, elle s'écrit :

```
Fonction F(x : Reel) : Reel
Debut
    Retourner 2 * x + 1
Fin
```

L'appel $F(3)$ vaut 7, exactement comme $f(3) = 7$ en mathématiques.

PROPRIÉTÉ Syntaxe générale d'une fonction

La déclaration d'une fonction obéit à la syntaxe suivante :

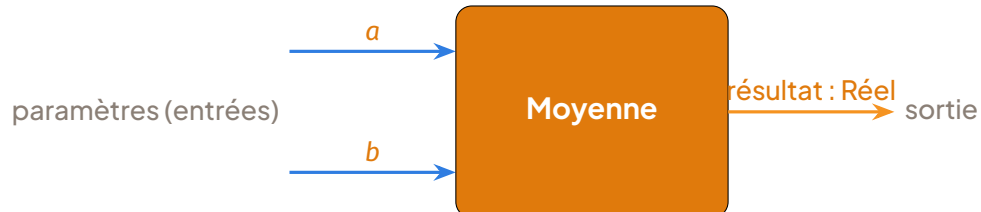
```
Fonction NomFonction(param1 : Type1, param2 : Type2, ...) : TypeResultat
    Var
        // variables locales (facultatives)
Debut
    // instructions du traitement
    Retourner expression
Fin
```

L'en-tête de la fonction (première ligne) précise trois choses : le nom de la fonction, la liste des paramètres formels avec leur type, et le type du résultat retourné (après le deux-points final). Le corps contient le traitement et au moins une instruction `Retourner`.

PROPRIÉTÉ Une fonction retourne exactement une valeur

Une fonction retourne exactement une valeur, dont le type est déclaré dans l'en-tête. Dès que l'instruction `Retourner` est exécutée, la fonction se termine immédiatement : les instructions qui suivraient éventuellement ne sont jamais exécutées, et le contrôle revient à l'algorithme appelant, à l'endroit exact de l'appel. (Propriété admise à ce niveau ; elle se justifie par l'exemple : dans `M <-- Moyenne(12, 15)`, l'appel doit pouvoir être remplacé par une valeur unique, ici 13,5.)

Une fonction se comporte donc comme une boîte noire : on lui donne des données, elle rend un résultat, sans qu'on ait besoin de connaître le détail du calcul à chaque utilisation.



Vision « boîte noire » d'une fonction : les paramètres *a* et *b* entrent, un seul résultat typé sort.

3.5 Appel d'une fonction dans l'algorithme principal

Déclarer une fonction ne l'exécute pas : il faut l'**appeler**. L'appel d'une fonction est une expression : il s'écrit avec le nom de la fonction suivi des **arguments** (les valeurs concrètes transmises) entre parenthèses, et il vaut le résultat retourné. On peut donc l'utiliser partout où une valeur est attendue : dans une affectation, dans un calcul, dans un affichage.

EXEMPLE La fonction Moyenne et son appel

```
Fonction Moyenne(a : Reel, b : Reel) : Reel
```

```
Debut
```

```
  Retourner (a + b) / 2
```

```
Fin
```

```
Algorithme Bulletin
```

```
Var
```

```
  S1, S2, M : Reel
```

```
Debut
```

```
  Ecrire("Note sequence 1 : ")
```

```
  Lire(S1)
```

```
  Ecrire("Note sequence 2 : ")
```

```
  Lire(S2)
```

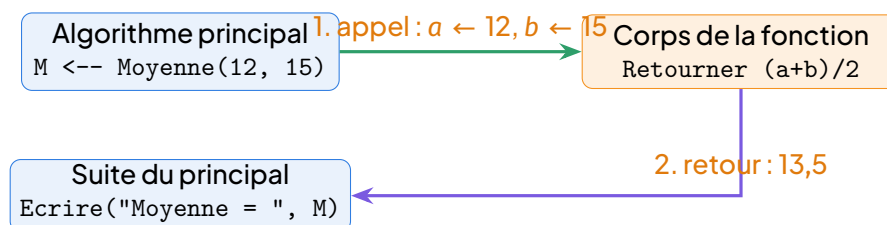
```

M <-- Moyenne(S1, S2) // appel : M reçoit le resultat
Ecrire("Moyenne = ", M)
Fin

```

Si l'utilisateur saisit 12 et 15, l'appel `Moyenne(S1, S2)` vaut 13,5 et cette valeur est affectée à M. Remarque que l'on aurait pu écrire directement `Ecrire("Moyenne = ", Moyenne(S1, S2))` : l'appel est une expression comme une autre.

Que se passe-t-il exactement lors de l'appel ? L'exécution de l'algorithme principal est suspendue, les arguments sont transmis aux paramètres, le corps de la fonction s'exécute, puis l'instruction `Retourner` ramène le résultat et l'exécution du principal reprend là où elle s'était arrêtée.



Flux d'exécution : le principal appelle la fonction (1), celle-ci calcule puis retourne le résultat (2), et le principal reprend son cours.

3.6 Méthode : écrire une fonction en quatre étapes

MÉTHODE Concevoir une fonction

Pour écrire correctement une fonction, suis toujours ces quatre étapes, dans l'ordre :

1. Identifier le résultat et son type. Pose-toi la question : « que doit produire la fonction ? » Une moyenne ? Un maximum ? Une mention ? Le type du résultat (Réel, Entier, Chaîne, Booléen) en découle.
2. Identifier les données nécessaires. De quoi la fonction a-t-elle besoin pour calculer ce résultat ? Chaque donnée devient un paramètre formel, avec son type.
3. Écrire l'en-tête. Choisis un nom explicite, liste les paramètres typés entre parenthèses, puis indique le type du résultat après le deux-points.
4. Écrire le corps. Déclare si besoin des variables locales, écris le traitement, et termine par `Retourner` suivi de l'expression résultat, du type annoncé.

Appliquons cette méthode à un exemple complet.

EXERCICE RÉSOLU 3 Maximum de trois notes

Énoncé. Écrire une fonction `Max2` qui retourne le plus grand de deux réels, puis un algorithme principal qui lit trois notes et affiche la plus grande en utilisant `Max2`.

SOLUTION

Solution détaillée.

Étape 1. Le résultat est le maximum de deux réels : c'est un Réel. Étape 2. La fonction a besoin des deux nombres à comparer : deux paramètres `a` et `b` de type Réel. Étapes 3 et 4. En-tête et corps :

```
Fonction Max2(a : Reel, b : Reel) : Reel
Debut
  Si a >= b Alors
    Retourner a
  Sinon
    Retourner b
FinSi
Fin
```

Remarque que l'instruction `Retourner` termine immédiatement la fonction : si `a >= b`, la fonction rend `a` et s'arrête ; le `Sinon` n'est atteint que dans le cas contraire.

Pour trois notes n_1, n_2, n_3 , l'astuce consiste à composer les appels : le maximum des trois est le maximum entre n_1 et le maximum de n_2 et n_3 :

```
Algorithme MaxiTroisNotes
Var
  N1, N2, N3, M : Reel
Debut
  Ecrire("Entrez trois notes : ")
  Lire(N1, N2, N3)
  M <-- Max2(N1, Max2(N2, N3)) // appels imbriqués
  Ecrire("La plus grande note est : ", M)
Fin
```

Vérification chiffrée. Avec $N_1 = 11$, $N_2 = 14$, $N_3 = 9$: l'appel interne `Max2(14, 9)` retourne 14 ; l'appel externe devient `Max2(11, 14)` qui retourne 14. L'algorithme affiche bien 14. Le même traitement « comparer deux valeurs » a été réutilisé deux fois sans être réécrit : c'est tout l'intérêt du sous-algorithme.

ATTENTION Les deux erreurs classiques avec les fonctions

- Oublier l'instruction `Retourner`. Sans `Retourner`, la fonction ne produit aucune valeur : l'appel `M <-- Moyenne(12, 15)` affecte à `M` un résultat indéfini. Vérifie toujours que tous les chemins d'exécution (le `Si` comme le `Sinon`) aboutissent à un `Retourner`.

- Afficher le résultat dans la fonction au lieu de le retourner. Écrire `Ecrire((a+b)/2)` dans la fonction affiche certes la valeur à l'écran, mais ne la transmet pas à l'appelant : impossible ensuite de s'en servir dans un calcul (par exemple pour comparer les moyennes de deux élèves). Règle d'or : une fonction calcule et retourne ; c'est l'algorithme principal qui décide d'afficher ou non.

EXERCICE 3 À toi de jouer

1. Écris une fonction `Mention(m : Reel) : Chaine` qui retourne "Admis" si $m \geq 10$ et "Ajourné" sinon, puis un algorithme principal qui lit la moyenne d'un élève et affiche sa mention.
2. Écris une fonction `Carre(x : Reel) : Reel` qui retourne x^2 , puis utilise-la pour afficher la somme des carrés de deux nombres saisis au clavier.

CORRIGÉ Exercice

Fonction `Mention(m : Reel) : Chaine`

Debut

Si $m \geq 10$ Alors

Retourner "Admis"

Sinon

Retourner "Ajourne"

FinSi

Fin

Algorithme Decision

Var

Moy : Reel

Debut

Ecrire("Moyenne de l'eleve : ")

Lire(Moy)

Ecrire("Mention : ", Mention(Moy))

Fin

Fonction `Carre(x : Reel) : Reel`

Debut

Retourner $x * x$

Fin

Algorithme SommeCarres

```
Var
  A, B, S : Reel
Debut
  Ecrire("Deux nombres : ")
  Lire(A, B)
  S <-- Carre(A) + Carre(B) // deux appels dans un calcul
  Ecrire("Somme des carres = ", S)
Fin
```

À RETENIR L'essentiel sur les fonctions

2. Un **sous-algorithme** est un bloc nommé et réutilisable ; il évite les répétitions, clarifie l'algorithme et facilite le travail en équipe.
 - Il existe deux familles : les **fonctions** (qui retournent une valeur) et les **procédures** (qui n'en retournent pas).
 - L'en-tête d'une fonction annonce son nom, ses paramètres formels typés et le type de son résultat.
 - **Retourner** transmet le résultat et termine immédiatement la fonction ; une fonction retourne exactement une valeur.
 - L'appel d'une fonction est une expression : il peut figurer dans une affectation, un calcul ou un affichage.

Nous savons désormais écrire et appeler des fonctions. Mais que se passe-t-il lorsqu'un sous-algorithme doit réaliser une action sans produire de valeur à récupérer — par exemple afficher un bulletin complet ou modifier plusieurs variables ? C'est le travail des **procédures**, que nous étudions dans la section suivante, avant de préciser les notions de variables locales, globales et de passage de paramètres.

4 LES PROCÉDURES

4.1 Transition : du calcul à l'action

Dans la section précédente, nous avons découvert les **fonctions** : des sous-algorithmes conçus pour calculer et retourner une valeur unique, utilisable ensuite dans une expression (affectation, test, affichage). C'est l'outil idéal pour factoriser un calcul complexe — par exemple, calculer la moyenne d'un tableau ou vérifier si un nombre est premier.

Mais tous les problèmes ne se résolvent pas par un calcul unique. Imaginez que vous deviez afficher la fiche complète d'un élève (nom, prénom, notes, moyenne, mention), ou saisir ses informations au clavier, ou encore trier un tableau sur place. Ces actions ne « retournent » pas une

valeur exploitable dans une formule ; elles modifient l'état du programme (l'écran, la mémoire, un fichier). C'est précisément le rôle des **procédures**.

LESAVAIS-TU ? Procédure vs Fonction : une histoire de vocabulaire

Dans les premiers langages (Fortran, Pascal, Ada, C), la distinction est syntaxique et stricte : **FUNCTION** retourne une valeur, **PROCEDURE** (ou **VOID** en C) n'en retourne pas. En Python, JavaScript ou Java modernes, on utilise le même mot-clé (**def**, **function**) pour les deux ; la différence devient sémantique : une « procédure » est simplement une fonction qui ne fait pas de **return** explicite (ou retourne **None/undefined**). En algorithmique (et au Probatoire), nous conservons la distinction formelle : **Fonction** = valeur de retour, **Procédure** = action / effet de bord.

4.2 Définition et syntaxe d'une procédure

DÉFINITION Procédure

Une procédure est un sous-algorithme nommé qui réalise une tâche ou une action (affichage, saisie, modification de variables passées par référence, écriture dans un fichier) sans retourner de valeur au programme appelant. Son appel constitue une instruction à part entière, et non une expression.

PROPRIÉTÉ Syntaxe générale (pseudo-code officiel)

La déclaration suit le schéma :

```
Procédure NomProcédure(Paramètres formels)
    // Déclarations locales (variables, constantes, types)
Début
    // Séquence d'instructions réalisant l'action
Fin
```

- **NomProcédure** : identifiant significatif, souvent un verbe d'action (Afficher, Saisir, Trier, Initialiser, MettreAJour).
- **Paramètres formels** : liste des entrées nécessaires (séparées par des virgules), chacune avec son type. Une procédure peut n'avoir aucun paramètre : `Procédure AfficherMenu()`.
- **Corps** : instructions algorithmiques classiques (affectations, structures de contrôle, appels d'autres sous-algorithmes).
- **Absence de Retourner** : c'est la marque distinctive. On sort de la procédure en atteignant **Fin** (un **Return** vide est toléré pour une sortie anticipée, mais en pseudo-code

strict on privilégie une structure à une seule sortie).

EXEMPLE Procédure d'affichage d'un enregistrement

Reprenons le type `Eleve` vu précédemment.

```
Type Eleve = Enregistrement
  Nom : Chaîne
  Prenom : Chaîne
  Notes : Tableau[1..5] de Réel
  Moyenne : Réel
Fin Enregistrement

Procédure AfficherEleve(E : Eleve)
  // Affiche la fiche complète d'un élève sur la console
  Variable i : Entier
  Début
    Ecrire(" --- FICHE ÉLÈVE ---")
    Ecrire("Nom : ", E.Nom)
    Ecrire("Prénom : ", E.Prenom)
    Ecrire("Notes : ")
    Pour i <- 1 à 5 Faire
      Ecrire(" Note ", i, " : ", E.Notes[i])
    Fin Pour
    Ecrire("Moyenne : ", E.Moyenne : 0 : 2)
    Si E.Moyenne >= 10 Alors
      Ecrire("Mention : ADMIS")
    Sinon
      Ecrire("Mention : RECALÉ")
    Fin Si
    Ecrire("-----")
  Fin
```

Remarquez : pas de `Retourner`. La procédure produit un effet visible (l'affichage) mais ne fournit aucune valeur à stocker dans une variable.

4.3 Appel d'une procédure : une instruction, pas une expression

C'est le point le plus important à retenir pour éviter les erreurs de compilation ou d'exécution.

PROPRIÉTÉ Règle d'appel

L'appel d'une procédure s'écrit comme une instruction simple, terminée par un point-

virgule (ou passage à la ligne). On écrit :

```
NomProcédure(Arguments effectifs) ;
```

Il est interdit d'écrire :

```
Variable <- NomProcédure(...) OU Si NomProcédure(...) Alors ...
```

ATTENTION Piège classique : affectation d'un appel de procédure

```
// ≡ ERREUR : Une procédure ne retourne RIEN (pas même 0 ou "vide").
```

```
// Le code suivant est syntaxiquement invalide en algorithmique strict.
```

```
Résultat <- AfficherEleve(MonEleve)
```

```
// ≡ CORRECT : Appel en tant qu'instruction.
```

```
AfficherEleve(MonEleve)
```

```
// La suite de l'algorithme continue ici.
```

Si vous avez besoin de récupérer un code de retour (succès/échec), transformez la procédure en fonction retournant un Booléen ou un Entier (code d'erreur).

4.4 Choisir entre Fonction et Procédure : la méthode

MÉTHODE Comment choisir ? La question magique

Face à un sous-problème à isoler, posez-vous cette unique question :

« Ai-je besoin d'un résultat unique à réutiliser dans un calcul, un test ou un affichage ultérieur ? »

1. OUI → C'est une **Fonction**. Elle Retourne une valeur. Exemples : CalculerMoyenne, EstMajeur, RechercherIndice, LireEntierPositif.
2. NON (l'objectif est d'afficher, saisir, modifier une structure passée en paramètre, écrire dans un fichier, dessiner) → C'est une **Procédure**. Elle agit par effet de bord. Exemples : AfficherMenu, SaisirEleve, TrierTableau, Permuter, InitialiserGrille.

FONCTION

But : Calculer, transformer, sélectionner → Produire une valeur

Mot-clé retour : Retourner
<expression> (obligatoire)

PROCÉDURE

Appel : Dans une expression

But : Afficher, Saisir, Modifier, Trier, Écrire → Produire un effet

Exemples types : Max, Racine,

Mot-clé retour : **Aucun**
(fin par Fin ou Return vide)

Appel : Une instruction seule
AfficherEleve(E);
TrierBulles(T, n);

Exemples types : AfficherTableau,
SaisirNotes, Permuter,
SauvegarderFichier

Tableau comparatif : Fonction vs Procédure — Critères de choix et syntaxe d'appel

4.5 Procédures avec paramètres : entrée, sortie, entrée-sortie

Comme les fonctions, les procédures communiquent avec l'extérieur via des **paramètres for-**

mels. La distinction passage par valeur / passage par référence (ou variable) est cruciale pour les procédures, car c'est souvent le seul moyen de « retourner » un résultat (modification d'une variable de l'appelant).

PROPRIÉTÉ Mécanismes de passage des paramètres

- Passage par valeur (par défaut pour types simples) : La procédure reçoit une copie. Toute modification du paramètre formel ne modifie pas l'argument effectif. Utilisé pour les paramètres d'entrée (ex : `n` dans `AfficherTableau(T, n)`).
- Passage par référence (variable) : La procédure travaille sur l'adresse de l'argument effectif. Toute modification modifie l'original. Indispensable pour les paramètres de sortie ou entrée-sortie (ex : le tableau `T` dans `TrierTableau(T, n)`, ou une variable `Somme` dans `CalculerSomme(T, n, Somme)`).
- Notation en pseudo-code : Souvent implicite (tableaux/structures = référence ; entiers/réels = valeur). Pour forcer la référence sur un type simple, on préfixe par `Variable:Procédure` `Permuter(Variable A, Variable B : Entier)`.

EXEMPLE Procédure de saisie (paramètre de sortie) et de tri (entrée-sortie)

```
// Paramètre E est de SORTIE : la procédure le remplit.
Procédure SaisirEleve(Variable E : Eleve)
  Variable i : Entier
  Début
    Ecrire("Nom : "); Lire(E.Nom)
    Ecrire("Prénom : "); Lire(E.Prenom)
    Pour i <- 1 à 5 Faire
      Ecrire("Note ", i, " : "); Lire(E.Notes[i])
    Fin Pour
    // Calcul interne de la moyenne
    E.Moyenne <- 0
    Pour i <- 1 à 5 Faire
      E.Moyenne <- E.Moyenne + E.Notes[i]
    Fin Pour
    E.Moyenne <- E.Moyenne / 5
  Fin

// Paramètre T est ENTRÉE-SORTIE : on le trie SUR PLACE.
Procédure TrierBulles(Variable T : Tableau[1..N] de Entier, n : Entier)
  Variable i, j, temp : Entier
  Début
    Pour i <- 1 à n-1 Faire
      Pour j <- 1 à n-i Faire
        Si T[j] > T[j+1] Alors
```

```

temp <- T[j]
T[j] <- T[j+1]
T[j+1] <- temp
Fin Si
Fin Pour
Fin Pour
Fin

```

4.6 Exemple guidé complet : Gestion de 3 élèves

Voici un algorithme principal structuré uniquement avec des procédures (et une fonction pour la moyenne, pour montrer la cohabitation).

EXERCICE RÉSOLU 4 Algorithme principal de gestion d'une mini-promotion

Énoncé : Écrire un algorithme qui gère 3 élèves. Pour chaque élève : saisie des infos (nom, prénom, 3 notes), calcul de la moyenne, affichage de la fiche. À la fin, afficher la meilleure moyenne de la promo.

Analyse :

- **Type** Eleve (enregistrement).
- **Tableau** Promo[1..3] de Eleve.
- **Procédure** SaisirEleve (remplit un élève).
- **Fonction** CalculerMoyenne (retourne un réel à partir de 3 notes).
- **Procédure** AfficherEleve (affiche un élève).
- **Fonction** MeilleureMoyenne (parcourt le tableau, retourne le max).

SOLUTION

Solution détaillée (Pseudo-code) :

```

Algorithme GestionPromo
// --- TYPES ---
Type Eleve = Enregistrement
  Nom : Chaîne
  Prenom : Chaîne
  Notes : Tableau[1..3] de Réel
  Moyenne : Réel
Fin Enregistrement

// --- SOUS-ALGORITHMES ---

// FONCTION : Calcule et RETOURNE la moyenne de 3 notes

```

```
Fonction CalculerMoyenne(N1, N2, N3 : Réel) : Réel
```

```
Début
```

```
    Retourner (N1 + N2 + N3) / 3
```

```
Fin
```

```
// PROCÉDURE : Remplit l'enregistrement E (paramètre de SORTIE)
```

```
Procédure SaisirEleve(Variable E : Eleve)
```

```
    Variable i : Entier
```

```
Début
```

```
    Ecrire(" --- SAISIE ÉLÈVE ---")
```

```
    Ecrire("Nom : "); Lire(E.Nom)
```

```
    Ecrire("Prénom : "); Lire(E.Prenom)
```

```
    Pour i <- 1 à 3 Faire
```

```
        Repeter
```

```
            Ecrire("Note ", i, " (0-20) : "); Lire(E.Notes[i])
```

```
        JusquA (E.Notes[i] >= 0) ET (E.Notes[i] <= 20)
```

```
    Fin Pour
```

```
// Appel de la FONCTION pour calculer la moyenne
```

```
E.Moyenne <- CalculerMoyenne(E.Notes[1], E.Notes[2], E.Notes[3])
```

```
Fin
```

```
// PROCÉDURE : Affiche l'élève E (paramètre d'ENTRÉE)
```

```
Procédure AfficherEleve(E : Eleve)
```

```
Début
```

```
    Ecrire("=====")
```

```
    Ecrire(E.Prenom, " ", E.Nom)
```

```
    Ecrire("Notes : ", E.Notes[1], " | ", E.Notes[2], " | ", E.Notes[3])
```

```
    Ecrire("Moyenne : ", E.Moyenne : 0 : 2)
```

```
    Si E.Moyenne >= 10 Alors Ecrire("Statut : ADMIS")
```

```
    Sinon Ecrire("Statut : RECALÉ")
```

```
    Fin Si
```

```
Fin
```

```
// FONCTION : Retourne la meilleure moyenne du tableau (ENTRÉE)
```

```
Fonction MeilleureMoyenne(Promo : Tableau[1..3] de Eleve) : Réel
```

```
    Variable i : Entier; Max : Réel
```

```
Début
```

```
    Max <- Promo[1].Moyenne
```

```
    Pour i <- 2 à 3 Faire
```

```
        Si Promo[i].Moyenne > Max Alors Max <- Promo[i].Moyenne Fin Si
```

```
    Fin Pour
```

```
    Retourner Max
```

```
Fin
```

```
// --- ALGORITHME PRINCIPAL ---
```

```
Variable Promo : Tableau[1..3] de Eleve
```

```
Variable i : Entier
```

```

Début
  Ecrire("=== GESTION MINI-PROMO (3 élèves) ===")

  // 1. Saisie des 3 élèves (appel PROCÉDURE)
  Pour i <- 1 à 3 Faire
    SaisirEleve(Promo[i]) // Modifie Promo[i] sur place
  Fin Pour

  Ecrire("", "\n=== BULLETINS ===")

  // 2. Affichage (appel PROCÉDURE)
  Pour i <- 1 à 3 Faire
    AfficherEleve(Promo[i]) // Lecture seule
  Fin Pour

  // 3. Résultat global (appel FONCTION dans expression)
  Ecrire("", "\n>>> Meilleure moyenne de la promo : ", MeilleureMoyenne(Promo) : 0 :2)
Fin

```

4.7 Visualisation du flux d'exécution : appel de procédure

Comprendre la pile d'exécution (call stack) est essentiel pour le débogage.

Contexte principal : variables globales au programme, E1 visible

```

...
AfficherEleve(E1) ; ← APPEL
... suite ...
Fin

```

Transfert de contrôle

**PROCÉDURE Aff-
cherEleve(E : Eleve)**

```

Début
  Ecrire("Nom : ", E.Nom)
  Ecrire("Moyenne
: ", E.Moyenne)

```

Contexte appelé : paramètre E (alias de E1), variables locales

Retour au caller — Pas de valeur retournée

Schéma du flux d'exécution lors de l'appel d'une procédure : suspension du principal, exécution de la procédure, retour au point d'appel suivant.

4.8 Exemple résolu : Procédure de filtrage et affichage

EXERCICE RÉSOLU 5 Procédure AfficherSupMoyenne

Énoncé : Écrire une procédure `AfficherSupMoyenne` qui reçoit un tableau `T` de `N` réels et sa taille `N`, calcule la moyenne des éléments, puis affiche uniquement les éléments strictement supérieurs à cette moyenne, un par ligne, avec leur indice.

Analyse :

- Entrées : `T` (tableau, lecture seule), `N` (entier, lecture seule).
- Sortie : Aucune (affichage direct sur console).
- Besoin interne : Calculer la moyenne → variable locale `Somme`, `Moy`.
- Deux parcours nécessaires : 1) `Somme` pour la moyenne. 2) Test et affichage.

SOLUTION

Algorithme de la procédure :

```
Procédure AfficherSupMoyenne(T : Tableau[1..N] de Réel, N : Entier)
```

```
    // Affiche les éléments du tableau T supérieurs à la moyenne
```

```
    Variable i : Entier
```

```
    Variable Somme, Moy : Réel
```

```
    Début
```

```
        // 1. Calcul de la moyenne (parcours 1)
```

```
        Somme ← 0
```

```
        Pour i ← 1 à N Faire
```

```
            Somme ← Somme + T[i]
```

```
        Fin Pour
```

```
        Si N > 0 Alors
```

```
            Moy ← Somme / N
```

```
        Sinon
```

```
            Moy ← 0 // Cas tableau vide
```

```
        Fin Si
```

```
        Ecrire("--- Éléments > Moyenne (", Moy : 0:2, ") ---")
```

```
        // 2. Filtrage et affichage (parcours 2)
```

```
        Pour i ← 1 à N Faire
```

```
            Si T[i] > Moy Alors
```

```
                Ecrire("T[", i, "] = ", T[i] : 0:2)
```

```
            Fin Si
```

```
        Fin Pour
```

```
        Ecrire("-----")
```

```
    Fin
```

Test dans un principal :

```

|  Algorithme TestFiltrage
|  Variable Notes : Tableau[1..6] de Réel
|  Variable i : Entier
|  Début
|      // Initialisation fixe pour test reproductible
|      Notes[1] <- 12; Notes[2] <- 8; Notes[3] <- 15
|      Notes[4] <- 10; Notes[5] <- 9; Notes[6] <- 14
|
|      // Appel de la procédure (instruction seule)
|      AfficherSupMoyenne(Notes, 6)
|
|      // Moyenne = (12+8+15+10+9+14)/6 = 68/6 = 11.33
|      // Affichés : T[1]=12, T[3]=15, T[6]=14
|  Fin

```

4.9 Synthèse des bonnes pratiques

À RETENIR Check-list « Procédure bien écrite »

1. Nom verbe d'action : Afficher..., Saisir..., Trier..., Initialiser..., Sauvegarder....
2. Commentaire d'en-tête : Rôle, paramètres (Entrée/Sortie/Entrée-Sortie), préconditions.
3. Paramètres typés : Préciser `Variable` pour les paramètres de sortie/entrée-sortie (tableaux, enregistrements, ou types simples modifiés).
4. Variables locales : Déclarer toutes les variables utilisées uniquement à l'intérieur (indices de boucle, accumulateurs, temporaires). Jamais de variable globale inutile.
5. Pas de Retourner avec valeur : La fin s'atteint par `Fin` (un `Return` vide pour sortie anticipée est toléré mais déconseillé en algo strict).
6. Appel correct : `NomProc(Args)` ; (instruction isolée).

EXPÉRIENCE TP Guidé : Module de gestion de stock (Procédures uniquement)

Objectif : Structurer un algorithme de mini-gestion de stock en utilisant exclusivement des procédures (et une fonction pour la valorisation). Contexte : Une boutique « Au Bon Prix » à Douala gère 5 articles. Chaque article a un code (entier), un libellé (chaîne), un prix unitaire (réel) et une quantité en stock (entier).

Matériel : Papier / éditeur de pseudo-code (Algobox, PSEInt, ou papier).

Étapes :

1. **Type** : Déclarer `Article` (Enregistrement) et `Stock` (Tableau[1..5] de `Article`).
2. Procédure `InitialiserStock`(Variable `S` : `Stock`) : Remplit le tableau avec des données « en dur » (3 articles renseignés, 2 vides : `Code=0`).
3. Procédure `AfficherStock`(`S` : `Stock`) : Affiche un joli tableau formaté (`Code` | `Libellé` | `Prix` | `Qté` | `Valeur`). Ignore les articles `Code=0`.
4. Procédure `SaisirArticle`(Variable `A` : `Article`) : Saisie contrôlée (`Prix > 0`, `Qté >= 0`).
5. Procédure `AjouterArticle`(Variable `S` : `Stock`) : Cherche le premier emplacement libre (`Code=0`), appelle `SaisirArticle` pour le remplir.
6. Fonction `ValorisationTotale`(`S` : `Stock`) : Réel : Somme (`Prix * Qté`) pour tout le stock.
7. Principal : Menu boucler (1. Afficher, 2. Ajouter, 3. Valorisation, 0. Quitter) appelant les procédures/fonction.

Observations attendues :

- Le principal reste très court (juste le menu et les appels).
- `AjouterArticle` modifie `S` → paramètre Variable.
- `AfficherStock` ne modifie rien → paramètre simple (entrée).
- La valorisation est une fonction car on a besoin du résultat pour l'afficher : `Ecrire("Total : ", ValorisationTotale(MonStock))`.

LESAVAIS-TU ? Le mot-clé `void` en langage C

En langage C (au programme de Première TI pour la programmation procédurale), une procédure se déclare avec le type de retour `void` (vide).

```
// Déclaration (prototype)
void afficherEleve(Eleve e); // Passage par valeur (copie)
void saisirEleve(Eleve *e); // Passage par adresse (pointeur) pour modifier l'original
void trierBulles(int t[], int n); // Tableau = passage par adresse implicite

// Définition
void afficherEleve(Eleve e) {
    printf("Nom : %s\n", e.nom);
    // ... pas de return ;
}

// Appel dans main
Eleve e1;
```

```
saisirEleve(&e1); // Il faut passer l'adresse (&) pour la procédure de saisie
afficherEleve(e1); // Passage par valeur OK pour affichage
```

La rigueur sur les pointeurs (*, &) en C est l'équivalent bas-niveau de la distinction « Variable / Valeur » en pseudo-code.

4.10 Exercices d'entraînement (Probatoire style)

EXERCICE 4 Exercice 1 : Permutation classique

Écrire une procédure `Permuter(Variable A, Variable B : Entier)` qui échange les valeurs de ses deux paramètres. Écrire un algorithme principal qui lit deux entiers `X` et `Y`, les affiche, appelle `Permuter(X, Y)`, puis les réaffiche pour prouver l'échange.

CORRIGÉ Exercice 1

```
Procédure Permuter(Variable A, Variable B : Entier)
  Variable Temp : Entier
Début
  Temp <- A
  A <- B
  B <- Temp
Fin

Algorithme TestPermut
Variable X, Y : Entier
Début
  Ecrire("X : "); Lire(X)
  Ecrire("Y : "); Lire(Y)
  Ecrire("Avant : X=", X, " Y=", Y)
  Permuter(X, Y) // Appel procédure : modifie X et Y réels
  Ecrire("Après : X=", X, " Y=", Y)
Fin
```

Point clé : Sans `Variable` (passage par référence), `A` et `B` seraient des copies locales; `X` et `Y` resteraient inchangés.

EXERCICE 5 Exercice 2 : Saisie sécurisée d'une note

Écrire une procédure `SaisirNote(Variable Note : Réel)` qui demande une note à l'utili-

sateur tant que la valeur n'est pas comprise entre 0 et 20 (inclus). La note valide est stockée dans le paramètre `Note`.

CORRIGÉ Exercice 2

```

Procédure SaisirNote(Variable Note : Réel)
Début
  Repeter
    Ecrire("Entrez une note (0 à 20) : "); Lire(Note)
    Si (Note < 0) Ou (Note > 20) Alors
      Ecrire("Erreur : note hors limites. Recommencez.")
    Fin Si
  JusquA (Note >= 0) Et (Note <= 20)
Fin

```

Utilité : Factorise la validation de saisie. Le principal appelle simplement `SaisirNote(Notes[i])` dans une boucle.

EXERCICE 6 Exercice 3 : Procédure de recherche d'indice (alternative fonction)

On veut trouver l'indice de la première occurrence d'une valeur `V` dans un tableau `T[1..N]`.

1. Proposer une fonction `RechercherIndice(T, N, V) : Entier` retournant l'indice (ou 0 si absent).
2. Proposer une procédure `RechercherIndiceProc(T, N, V, Variable Pos : Entier)`.
3. Expliquer dans quel cas on préfère l'une ou l'autre.

CORRIGÉ Exercice 3

1. Fonction :

```

Fonction RechercherIndice(T : Tableau[1..N] de Entier, N, V : Entier) : Entier
  Variable i : Entier
Début
  Pour i <- 1 à N Faire
    Si T[i] = V Alors Retourner i Fin Si
  Fin Pour
  Retourner 0 // Non trouvé
Fin

```

2. Procédure (version stricte, sans Return anticipé) :

Procédure RechercherIndiceProc(T : Tableau[1..N] de Entier, N, V : Entier, Variable Pos : Entier)

Variable i : Entier

Variable Trouve : Booléen

Début

Pos ← 0

Trouve ← Faux

Pour i ← 1 à N Faire

Si (T[i] = V) ET (Non Trouve) Alors

Pos ← i

Trouve ← Vrai

Fin Si

Fin Pour

Fin

3. Choix :

- **Fonction si on utilise le résultat direct** : Si RechercherIndice(T, N, 5) > 0 Alors
.... C'est la forme canonique en algorithmique moderne.
- **Procédure si la recherche est une étape parmi d'autres dans un traitement complexe modifiant plusieurs variables de sortie, ou si le langage impose une seule sortie (Fin) pour les procédures.** En algo Probatoire, la fonction est préférable pour une recherche simple car plus expressive.

ATTENTION Derniers pièges pour le Probatoire

- **Confondre paramètre formel / effectif** : Dans Procédure P(X : Entier); P(A); X est le formel (local à P), A est l'effectif (variable du principal). Ils n'ont pas besoin de porter le même nom.
- **Oublier Variable pour un paramètre de sortie** : Si Procédure Saisir(N : Entier) (sans Variable), N est une copie locale. La saisie est perdue à la fin de la procédure. Sanction : 0 pt sur la question.
- **Écrire Return avec une valeur dans une procédure** : Interdit. Return seul (sortie anticipée) est toléré dans certains logiciels, mais Return Valeur transforme la procédure en fonction.
- **Appeler une procédure dans une expression** : Moy ← (AfficherEleve(E) + 10) → Erreur fatale.

5 VARIABLES GLOBALES, VARIABLES LOCALES, PARAMÈTRES

Dans la section précédente, nous avons appris à écrire des procédures et à les appeler depuis l'algorithme principal. Mais une question essentielle reste en suspens : comment les sous-algorithmes communiquent-ils avec le reste du programme ? Quand une procédure `AfficherMoyenne` calcule un résultat, comment l'algorithme principal le récupère-t-il ? Et inversement, comment transmettre à une fonction les données dont elle a besoin ? Deux mécanismes existent : les **variables globales**, accessibles partout, et les **paramètres**, qui servent de « sas » contrôlé entre l'appelant et le sous-algorithme. Bien comprendre la différence entre ces deux mécanismes — et savoir choisir le bon — est l'une des compétences les plus importantes de l'algorithmique, et une question classique du Probatoire.

5.1 Variables globales et variables locales

Intuition. Imagine une entreprise. Le tableau d'affichage du hall est visible par tous les employés, de tous les services : c'est une variable globale. En revanche, le carnet de notes du comptable reste dans son bureau ; personne d'autre ne peut le consulter, et quand il part en congé, son carnet ne sert à personne : c'est une variable locale. En algorithmique, la même logique s'applique selon l'endroit où la variable est déclarée.

DÉFINITION Variable globale

Une **variable globale** est une variable déclarée dans la partie déclarative de l'algorithme principal. Elle est visible et utilisable partout : dans l'algorithme principal et dans tous les sous-algorithmes (fonctions et procédures) qui lui appartiennent. Elle existe pendant toute la durée d'exécution du programme.

DÉFINITION Variable locale

Une **variable locale** est une variable déclarée à l'intérieur d'un sous-algorithme (fonction ou procédure), dans sa propre partie déclarative. Elle n'est visible et utilisable que dans ce sous-algorithme. Elle est créée au moment de l'appel et détruite à la fin de l'exécution du sous-algorithme : sa valeur est alors perdue.

DÉFINITION Portée d'une variable

La **portée** (ou visibilité) d'une variable est l'ensemble des endroits du programme où cette variable peut être utilisée. La portée d'une variable globale est le programme tout entier ; la portée d'une variable locale est limitée au corps du sous-algorithme où elle est déclarée.

Algorithme principal`g : Entier (globale)`*g est visible partout :
ici, dans F et dans P***Fonction F**`x : Entier
(locale à F)`**Procédure P**`y : Réel
(locale à P)`*x et y sont invisibles
hors de leur sous-algorithme*

Portée des variables : la globale *g* est accessible partout ; les locales *x* et *y* sont confinées dans leur sous-algorithme respectif.

PROPRIÉTÉ**Durée de vie des variables locales (admise)**

Une variable locale n'existe que pendant l'exécution de son sous-algorithme : elle est créée à l'appel et détruite au retour. Deux conséquences importantes :

- entre deux appels successifs d'une même fonction, une variable locale ne conserve pas sa valeur ;
- deux sous-algorithmes différents peuvent déclarer des variables locales de même nom sans aucun conflit : ce sont des variables distinctes, chacune dans sa propre « boîte ».

EXEMPLE**Trace pas à pas : portée en action**

Considérons l'algorithme suivant et suivons les valeurs à chaque étape.

Algorithme Portee
Variable
<code>g : Entier</code> // variable globale
Fonction Doubler() : Entier
Variable
<code>x : Entier</code> // locale à Doubler
Debut
<code>x <- g * 2</code> // g est accessible ici
Retourner x
Fin

```

| Debut
|   g <- 5
|   Ecrire(Doubler()) // affiche 10
|   Ecrire(g)         // affiche 5
|   Ecrire(x)         // ERREUR : x n'existe pas ici !
| Fin

```

Trace d'exécution : (1) `g` reçoit 5 ; (2) appel de `Doubler()` : la locale `x` est créée, reçoit $5 \times 2 = 10$, la fonction retourne 10 puis `x` est détruite ; (3) l'instruction `Ecrire(g)` est correcte car `g` est globale ; (4) `Ecrire(x)` provoque une erreur : `x` est hors de sa portée.

5.2 Paramètres formels et paramètres effectifs (arguments)

Intuition. Une fonction comme `Moyenne` doit pouvoir calculer la moyenne de n'importe quelles deux notes, pas seulement de deux variables fixes du programme. Il faut donc un mécanisme pour transmettre des valeurs au moment de l'appel : ce sont les paramètres. L'en-tête du sous-algorithme annonce des « cases vides » (les paramètres formels) ; l'appel les remplit avec des valeurs concrètes (les arguments).

DÉFINITION Paramètre formel

Un **paramètre formel** est un nom déclaré dans l'en-tête d'un sous-algorithme, accompagné de son type. Il joue le rôle d'une variable locale recevant une valeur au moment de l'appel. Exemple : dans `Fonction Moyenne(a, b : Reel) : Reel`, les paramètres formels sont `a` et `b`.

DÉFINITION Paramètre effectif (argument)

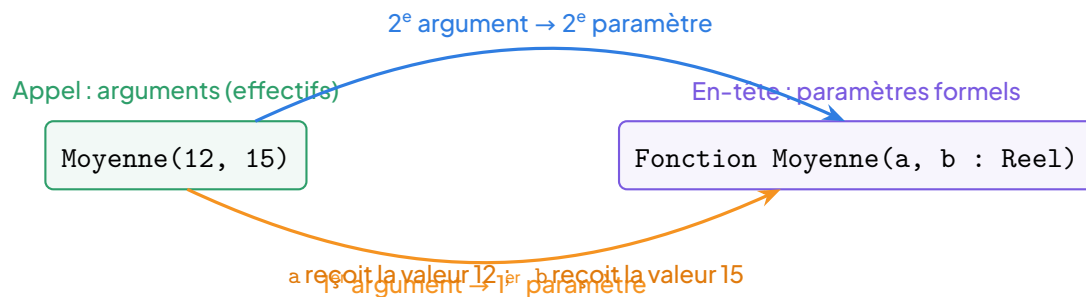
Un **paramètre effectif**, appelé aussi **argument**, est la valeur ou la variable concrète transmise au sous-algorithme lors de l'appel. Exemple : dans l'instruction `Ecrire(Moyenne(12, 15))`, les arguments sont 12 et 15.

PROPRIÉTÉ Correspondance lors de l'appel

Au moment de l'appel, les arguments remplacent les paramètres formels dans l'ordre : le premier argument est affecté au premier paramètre, le deuxième au deuxième, etc. Deux exigences sont impératives :

- même nombre : il doit y avoir exactement autant d'arguments que de paramètres formels ;
- compatibilité de type : le type de chaque argument doit correspondre au type du pa-

paramètre formel associé (un Reel ne peut pas être passé là où un Caractere est attendu).



Correspondance arguments ↔ paramètres formels lors de l'appel Moyenne(12, 15) : l'ordre détermine l'association.

EXEMPLE Code complet commenté

```

Algorithme Bulletins
Variable
    n1, n2 : Reel          // variables globales

Fonction Moyenne(a, b : Reel) : Reel
// a et b : paramètres formels (locaux à la fonction)
Debut
    Retourner (a + b) / 2
Fin

Debut
    Ecrire("Première note : ")
    Lire(n1)
    Ecrire("Deuxième note : ")
    Lire(n2)
    // n1 et n2 sont les arguments (paramètres effectifs)
    Ecrire("Moyenne = ", Moyenne(n1, n2))
Fin
    
```

Si l'élève saisit 12 puis 15, l'appel Moyenne(n1, n2) transmet les valeurs 12 et 15 : a vaut 12, b vaut 15, et la fonction retourne $(12 + 15)/2 = 13,5$.

5.3 Passage par valeur et passage par référence (complément utile au Bac)

LESAVAIS-TU ? Comment les arguments sont-ils transmis ?

Lorsqu'on passe une variable comme argument, deux stratégies existent selon les langages. Avec le **passage par valeur**, le sous-algorithme reçoit une copie de la valeur : même s'il modifie son paramètre, la variable d'origine reste inchangée. Avec le **passage par référence** (ou par adresse), le sous-algorithme travaille directement sur la variable d'origine : toute modification du paramètre modifie réellement la variable de l'appelant. C'est le mécanisme qui permet à une procédure d'« échanger » le contenu de deux variables. En pseudo-code, on signale souvent le passage par référence en faisant précéder le paramètre d'un mot-clé comme `Var : Procédure Echanger(Var x, Var y : Entier)`.

EXEMPLE Illustration des deux passages

```

Procédure ParValeur(n : Entier)    // passage par valeur
Debut
  n <- n + 10
Fin

Procédure ParReference(Var n : Entier) // passage par référence
Debut
  n <- n + 10
Fin

// Dans l'algorithme principal, avec t <- 5 :
// ParValeur(t)    : t vaut toujours 5 après l'appel
// ParReference(t) : t vaut 15 après l'appel

```

5.4 Exemple résolu complet et bonnes pratiques**EXERCICE RÉSOLU 6** Trace complète : variable globale et fonction

Énoncé. On considère l'algorithme suivant :

```

Algorithme Suivi
Variable
  compteur : Entier

Fonction Carre(n : Entier) : Entier
Variable
  r : Entier
Debut
  compteur <- compteur + 1

```

```

    r <- n * n
    Retourner r
Fin

Debut
    compteur <- 0
    Ecrire(Carre(3))
    Ecrire(Carre(5))
    Ecrire("Appels : ", compteur)
Fin

```

Donner la trace complète de l'exécution et ce qui est affiché.

SOLUTION

Solution détaillée. Dressons le tableau des valeurs étape par étape :

Étape	Instruction	compteur	n	r	Affichage
1	compteur <- 0	0	—	—	—
2	Appel Carre(3) : n reçoit 3	0	3	—	—
3	compteur <- compteur + 1	1	3	—	—
4	r <- n * n puis retour	1	3	9	9
5	Appel Carre(5) : n reçoit 5	1	5	—	—
6	compteur <- compteur + 1	2	5	—	—
7	r <- n * n puis retour	2	5	25	25
8	Ecrire("Appels : ", compteur)	2	—	—	Appels : 2

Le programme affiche donc successivement 9, 25 puis Appels : 2. Remarques clés : (a) la fonction accède à la globale `compteur` et la modifie durablement, car une globale survit à la fin du sous-algorithme ; (b) les locales `n` et `r` sont recrées à chaque appel : au deuxième appel, `r` ne « se souvient » pas qu'il valait 9 ; (c) c'est précisément parce que `compteur` est globale qu'elle peut cumuler le nombre d'appels.

ATTENTION Erreurs fréquentes à éviter absolument

- Utiliser une variable locale hors de son sous-algorithme : écrire `Ecrire(r)` dans l'algorithme principal alors que `r` est locale à `Carre` est une erreur de portée. Si tu as besoin du résultat, utilise la valeur retournée par la fonction.
- Inverser l'ordre des arguments : `Moyenne(15, 12)` donne ici le même résultat par chance, mais pour `Puissance(base, exposant)`, écrire `Puissance(2, 10)` au lieu de `Puissance(10, 2)` calcule 2^{10} au lieu de 10^2 ! L'ordre des arguments doit respecter l'ordre des paramètres formels.

- Nombre d'arguments incorrect : appeler `Moyenne(12)` ou `Moyenne(12, 15, 18)` alors que la fonction attend deux paramètres est une erreur. Compte toujours : autant d'arguments que de paramètres formels.
- Incompatibilité de type : passer la chaîne "douze" à un paramètre déclaré `Reel` est rejeté. Vérifie les types un par un.

MÉTHODE Bien choisir : globale ou paramètre ?

1. Par défaut, privilégie le passage par paramètres : un sous-algorithme qui ne dépend que de ses paramètres est autonome, testable isolément et réutilisable dans un autre programme.
2. Réserve les variables globales aux données réellement partagées par tout le programme (un compteur général, une configuration).
3. Documente l'en-tête de chaque sous-algorithme : rôle, paramètres attendus (types), valeur retournée.
4. Relis chaque appel en vérifiant trois points : nombre d'arguments, ordre, types.

LESAVAIS-TU ? Pourquoi les professionnels se méfient des globales

Dans les grands logiciels (des millions de lignes de code), l'abus de variables globales est l'une des causes majeures de bogues : n'importe quelle partie du programme peut modifier une globale, et il devient presque impossible de savoir « qui » a produit une valeur incorrecte. C'est pourquoi les langages modernes (Java, Python, C#) et les méthodes de développement professionnelles encouragent fortement les paramètres explicites : chaque donnée nécessaire à un sous-programme est visible dans son en-tête, ce qui rend le code lisible, prévisible et facile à tester. En adoptant dès maintenant cette discipline dans tes algorithmes, tu acquiers un réflexe de programmeur professionnel.

À RETENIR L'essentiel de la section

- Une **variable globale** est déclarée dans l'algorithme principal : visible partout, elle vit pendant tout le programme.
- Une **variable locale** est déclarée dans un sous-algorithme : visible uniquement là, créée à l'appel, détruite au retour.
- La **portée** est l'ensemble des endroits où une variable est utilisable.

- Les **paramètres formels** sont déclarés dans l'en-tête ; les **arguments** (paramètres effectifs) sont les valeurs concrètes de l'appel. La correspondance se fait dans l'ordre, avec même nombre et types compatibles.
- Passage **par valeur** : copie, l'originale est protégée ; passage **par référence** : le sous-algorithme peut modifier la variable de l'appelant.
- Bonne pratique : limiter les globales, préférer les paramètres pour des sous-algorithmes réutilisables.

EXERCICE 7 Portée et paramètres

Soit l'algorithme suivant :

```

Algorithme Test
Variable
  a : Entier
Fonction F(x : Entier) : Entier
Variable
  a : Entier
Debut
  a <- x + 1
  Retourner a * 2
Fin
Debut
  a <- 100
  Ecrire(F(4))
  Ecrire(a)
Fin
  
```

1. Quelles sont les variables globales ? Les variables locales ? Les paramètres formels et les arguments ?
2. Qu'affiche le programme ? Justifie par une trace d'exécution.
3. Pourquoi la variable `a` de l'algorithme principal n'est-elle pas modifiée par la fonction ?

CORRIGÉ Exercice : Portée et paramètres

1. Variable globale : `a` (déclarée dans l'algorithme principal). Variables locales : `x` (paramètre formel) et `a` (déclarée dans `F`). Paramètre formel : `x` ; argument effectif : la valeur 4 lors de l'appel `F(4)`.

2. Trace : a globale reçoit 100. Appel $F(4)$: x reçoit 4 ; la a locale reçoit $4+1 = 5$; la fonction retourne $5 \times 2 = 10$. Le programme affiche donc 10 puis 100.
3. La variable a déclarée dans F est une variable locale distincte de la globale de même nom : à l'intérieur de F , c'est la locale qui est utilisée (on dit qu'elle masque la globale). La globale a n'est jamais touchée par la fonction et conserve sa valeur 100.

6 SYNTHÈSE : STRUCTURER UN ALGORITHME COMPLET

Dans la section précédente, tu as appris à distinguer les **variables globales** des **variables locales**, et à faire circuler les données entre un algorithme et ses sous-algorithmes grâce aux **paramètres**. Tu possèdes maintenant toutes les briques : tableaux, enregistrements, fonctions, procédures, règles de portée. Il reste à répondre à la question que pose tout problème réel : comment assembler toutes ces briques en un algorithme complet, clair et correct ? C'est l'objet de cette synthèse, qui est aussi la compétence la plus souvent évaluée au Probatoire : on te donne une situation (gestion de notes, de stock, de facturation) et on te demande de produire ou de comprendre un algorithme structuré.

6.1 Démarche générale de résolution d'un problème

Face à un énoncé, le piège classique est de se jeter sur l'écriture du corps principal. Un bon programmeur fait exactement l'inverse : il réfléchit d'abord aux données, puis aux traitements, et n'écrit le programme principal qu'à la fin. C'est la démarche descendante : on part du problème global et on le découpe en sous-problèmes de plus en plus simples, jusqu'à obtenir des tâches que l'on sait coder directement.

MÉTHODE Les 4 étapes pour structurer un problème avec sous-algorithmes

1. Analyser les données. Identifier les informations manipulées : quelles sont les données d'entrée ? les résultats attendus ? Quelles grandeurs sont simples (un entier, un réel) et quelles grandeurs sont composées (une liste de valeurs, une fiche avec plusieurs champs) ?
2. Choisir les structures de données. Une collection homogène de valeurs $\Rightarrow$ un **tableau**. Une fiche regroupant des informations de types différents $\Rightarrow$ un **enregistrement**. Une collection de fiches $\Rightarrow$ un tableau d'enregistrements. Déclarer ces types et ces variables en global.
3. Découper en sous-algorithmes. Pour chaque tâche répétitive ou autonome (saisie, calcul, affichage, recherche), créer un sous-algorithme. Poser la question : la tâche retourne-t-elle un seul résultat ? Si oui $\Rightarrow$ **fonction**. Si elle effectue une action ou produit plusieurs effets (saisie, affichage, modification de plusieurs variables) $\Rightarrow$ **procédure**. Préciser pour chacun ses paramètres (formels) et ses variables locales.

4. Assembler. Écrire dans l'ordre : les déclarations de types, les variables globales, les sous-algorithmes, puis l'algorithme principal, qui doit rester court : il se contente d'appeler les sous-algorithmes dans le bon ordre.

PROPRIÉTÉ Ordre obligatoire des déclarations dans un algorithme structuré

Dans un algorithme utilisant des sous-algorithmes, l'ordre d'écriture est imposé :

1. déclaration des types (enregistrements) ;
2. déclaration des constantes et des variables globales (qui peuvent utiliser ces types) ;
3. définition des fonctions et procédures ;
4. corps de l'algorithme principal.

Règle d'or : toute entité doit être déclarée avant d'être utilisée. Un type enregistrement doit donc être déclaré avant le tableau qui l'utilise, et un sous-algorithme avant l'algorithme principal qui l'appelle.

PROPRIÉTÉ Passage des paramètres : paramètres formels et effectifs

- Les **paramètres formels** sont les variables déclarées dans l'en-tête du sous-algorithme (ex. `E : Eleve` dans `Fonction MoyenneEleve(E : Eleve)`). Ils existent seulement pendant l'exécution du sous-algorithme.
- Les **paramètres effectifs** (ou arguments) sont les expressions fournies lors de l'appel (ex. `Classe[j]` dans `MoyenneEleve(Classe[j])`).
- Au moment de l'appel, chaque paramètre effectif est évalué et sa valeur est transmise au paramètre formel correspondant (par ordre et par type). Dans le pseudo-code du Probatoire, le passage se fait par valeur pour les types simples (Entier, Reel, Booleen, Chaîne) et par référence (adresse) pour les types structurés (Tableau, Enregistrement) sauf précision contraire, ce qui permet à une procédure de modifier le contenu d'un tableau ou d'un enregistrement global passé en paramètre.

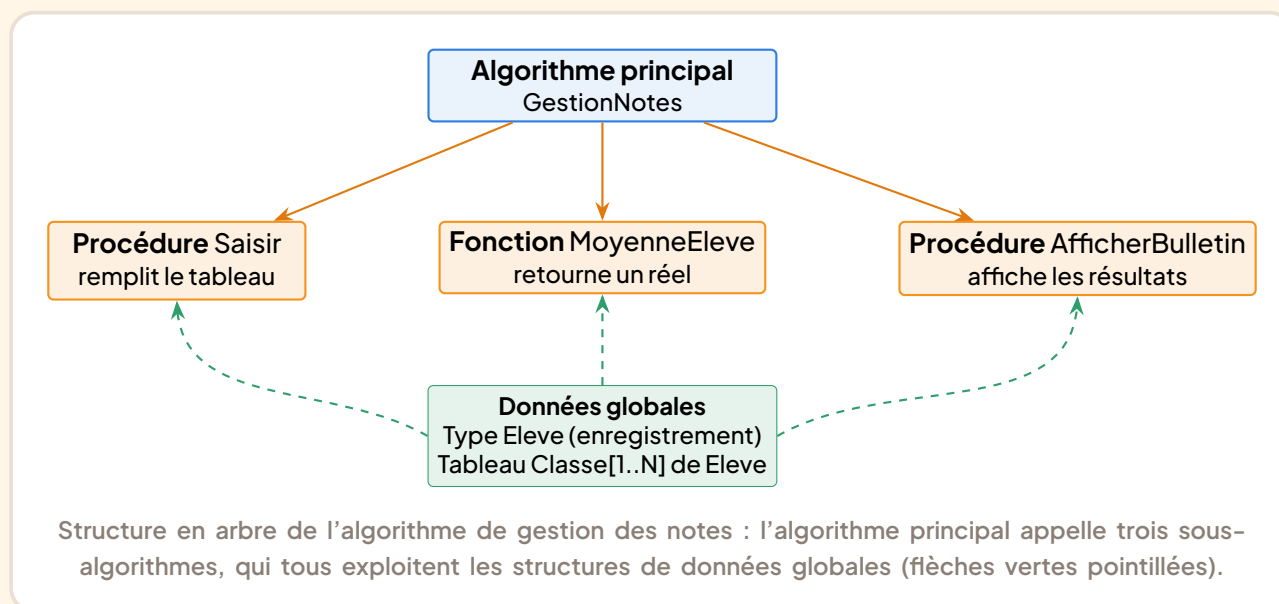
6.2 Architecture type d'un algorithme bien structuré

Un algorithme complet de gestion (notes, stock, bibliothèque) suit presque toujours la même architecture, qu'il faut savoir reconnaître et reproduire :

- **Déclarations globales** : les types enregistrement (par exemple `Eleve`), les constantes (par exemple `N = 5`) et les tableaux globaux (par exemple `Classe`) ;
- **Fonctions de calcul** : elles reçoivent des données en paramètres et retournent un résultat

(moyenne, total, maximum) sans rien afficher ni saisir ; elles sont pures (sans effet de bord) ;

- **Procédures d'entrée/sortie** : elles dialoguent avec l'utilisateur (saisie des données, affichage des résultats) et modifient les variables globales ou reçoivent les données par paramètres ; elles réalisent les effets de bord ;
- **Algorithme principal** : quelques lignes seulement, qui orchestrent les appels dans l'ordre logique (saisie → traitements → affichage).



6.3 Étude de cas guidée : gestion des notes d'une classe

Situation. Le proviseur d'un lycée de Yaoundé veut un programme qui, pour une classe de $N = 5$ élèves, saisit le nom de chaque élève et ses trois notes (sur 20), calcule la moyenne de chaque élève, puis affiche le bulletin récapitulatif de la classe.

Étape 1 — Analyse des données. Pour chaque élève, on manipule une fiche composée : un nom (chaîne) et trois notes (réels). Les cinq fiches forment une collection homogène.

Étape 2 — Structures de données. Une fiche élève $\Rightarrow$ enregistrement `Eleve`. L'ensemble des élèves $\Rightarrow$ tableau `Classe[1..N]` de `Eleve` (tableau d'enregistrements).

Étape 3 — Découpage. Trois tâches : la saisie (action, plusieurs effets $\Rightarrow$ procédure `Saisir`), le calcul d'une moyenne (un seul résultat $\Rightarrow$ fonction `MoyenneEleve`), l'affichage du bulletin (action $\Rightarrow$ procédure `AfficherBulletin`).

Étape 4 — Assemblage. Voici l'algorithme complet, commenté. Respecte scrupuleusement l'ordre des déclarations.

EXEMPLE Algorithme complet : gestion des notes de 5 élèves

```

Algorithme GestionNotes
// ----- 1. Declaration des types -----

```

```
Type
  Eleve = Enregistrement
  nom : Chaîne
  n1, n2, n3 : Reel    // les trois notes sur 20
Fin Enregistrement

// ----- 2. Constantes et variables globales -----
Constantes
  N = 5
Variables
  Classe : Tableau[1..N] de Eleve // tableau global d'enregistrements

// ----- 3. Fonction de calcul -----
Fonction MoyenneEleve(E : Eleve) : Reel
  // Parametre formel : E (un enregistrement Eleve)
  // Retourne la moyenne des trois notes de E
  Debut
    Retourner (E.n1 + E.n2 + E.n3) / 3
  Fin Fonction

// ----- 4. Procedure de saisie -----
Procedure Saisir()
  // Remplit le tableau global Classe (effet de bord controle)
  Variables
    j : Entier    // variable LOCALE a la procedure
  Debut
    Pour j de 1 a N Faire
      Ecrire("Nom de l'eleve ", j, " : ")
      Lire(Classe[j].nom)
      Ecrire("Note 1 : "); Lire(Classe[j].n1)
      Ecrire("Note 2 : "); Lire(Classe[j].n2)
      Ecrire("Note 3 : "); Lire(Classe[j].n3)
    Fin Pour
  Fin Procedure

// ----- 5. Procedure d'affichage -----
Procedure AfficherBulletin()
  // Affiche chaque eleve et sa moyenne (appelle MoyenneEleve)
  Variables
    j : Entier    // variable LOCALE
  Debut
    Ecrire("===== BULLETIN DE LA CLASSE =====")
    Pour j de 1 a N Faire
      Ecrire(Classe[j].nom, " : moyenne = ",
        MoyenneEleve(Classe[j]))
    Fin Pour
  Fin Procedure

// ----- 6. Algorithme principal (court !) -----
```

```

| Debut
|   Saisir()           // appel de la procedure de saisie
|   AfficherBulletin() // appel de la procedure d'affichage
| Fin

```

Observe bien l'appel `MoyenneEleve(Classe[j])` : le **paramètre effectif** (argument) est l'enregistrement `Classe[j]`, qui vient se substituer au **paramètre formel** `E` à chaque tour de boucle. La fonction est ainsi appelée 5 fois, avec 5 arguments différents, sans qu'on ait eu à réécrire le calcul. Remarque aussi que le compteur `j` est déclaré en local dans chaque procédure : les deux variables `j` sont distinctes et ne se perturbent pas. La variable globale `i` présente dans une version précédente a été supprimée car inutilisée (bonne pratique).

6.4 Trace d'exécution partielle

Pour vérifier qu'un algorithme est correct (et pour répondre aux questions de type « que contient la variable `X` après exécution ? »), on réalise une **trace d'exécution** : on simule à la main le déroulement du programme sur un petit jeu de données.

Supposons que l'utilisateur saisisse : Amina (12 ; 14 ; 10), puis Bruno (8 ; 9 ; 13), puis trois autres élèves. Traçons les deux premiers tours de la procédure `Saisir` et le début de `AfficherBulletin` :

Étape	Instruction	Effet	État après exécution
1	<code>j ← 1</code>	début du Pour de <code>Saisir</code>	<code>j = 1</code> (local à <code>Saisir</code>)
2	<code>Lire(Classe[1].nom)</code>	saisie du nom	<code>Classe[1].nom = « Amina »</code>
3	<code>Lire(Classe[1].n1..n3)</code>	saisie des notes	<code>Classe[1] = (Amina, 12, 14, 10)</code>
4	<code>j ← 2</code>	tour suivant	<code>Classe[2] = (Bruno, 8, 9, 13)</code>
5	<code>AfficherBulletin()</code>	appel depuis le programme principal	début de l'affichage
6	<code>MoyenneEleve(Classe[1])</code>	<code>E ← Classe[1]</code> ; retourne $(12 + 14 + 10)/3$	affiche « Amina : moyenne = 12 »
7	<code>MoyenneEleve(Classe[2])</code>	retourne $(8 + 9 + 13)/3$	affiche « Bruno : moyenne = 10 »

La trace confirme que chaque enregistrement est bien rempli avant d'être lu, que la fonction reçoit la bonne copie/référence de l'enregistrement, et que les indices du tableau restent dans l'intervalle $[1, N]$.

6.5 Lire et comprendre un algorithme à sous-algorithmes

Au Probatoire, on te donne souvent un algorithme complet et on te pose des questions de compréhension : quel est le rôle de la fonction `F` ? que vaut la variable `X` à la fin ? que se passe-t-il si on supprime l'appel à la procédure `P` ? Voici la méthode de lecture :

1. **Repérer les déclarations globales** : quels types, quels tableaux ? Cela donne immédiate-

ment la nature des données manipulées.

2. **Identifier chaque sous-algorithme** : fonction ou procédure ? paramètres formels ? valeur retournée ? Dédurre son rôle de son nom et de son corps.
3. **Lire l'algorithme principal en dernier** : il indique l'ordre des appels, donc la logique générale du programme.
4. **Faire une trace** sur un petit exemple si une question porte sur une valeur précise.

EXERCICE RÉSOLU 7**Questions de compréhension sur l'algorithme GestionNotes**

On considère l'algorithme `GestionNotes` ci-dessus. (1) Quel est le rôle de la fonction `MoyenneEleve` ? (2) La variable `j` de la procédure `Saisir` est-elle accessible dans `AfficherBulletin` ? (3) Que se passe-t-il si, dans l'algorithme principal, on inverse les deux appels ?

SOLUTION

(1) `MoyenneEleve` reçoit en paramètre un enregistrement de type `Eleve` et retourne un réel : la moyenne arithmétique de ses trois notes. C'est une fonction de calcul pur : elle ne saisit rien, n'affiche rien et ne modifie pas son paramètre.

(2) Non. La variable `j` est locale à la procédure `Saisir` : elle est créée à l'entrée de la procédure et détruite à la sortie. `AfficherBulletin` possède sa propre variable locale `j`, distincte. Seules les variables globales (`Classe`, `N`) sont partagées.

(3) Si l'on écrit `AfficherBulletin()` avant `Saisir()`, le bulletin sera affiché avec un tableau `Classe` non initialisé : les champs contiennent des valeurs indéterminées, et les moyennes calculées seront fausses (ou provoqueront une erreur d'exécution selon l'environnement). Moralité : dans un algorithme de gestion, la saisie doit toujours précéder les traitements et les affichages.

ATTENTION Erreurs fréquentes à l'examen

- Déclarer les sous-algorithmes après le corps principal. L'algorithme principal appelle `Saisir` et `MoyenneEleve` : ils doivent donc être définis avant le `Debut` principal. Un sous-algorithme placé après le `Fin` final est tout simplement invisible pour le compilateur/interpréteur.
- Oublier de déclarer le type enregistrement avant le tableau. Écrire `Classe : Tableau[1..N]` de `Eleve` alors que le type `Eleve` n'a pas encore été déclaré est une erreur de syntaxe : toute entité doit être déclarée avant son utilisation.
- Confondre paramètre formel et paramètre effectif. `E` est le paramètre formel (dans l'en-tête de la fonction) ; `Classe[j]` est l'argument effectif (à l'appel). Ne jamais utiliser le nom formel `E` dans l'algorithme principal.
- Utiliser une variable locale d'une procédure dans une autre. Le `j` de `Saisir` n'existe pas

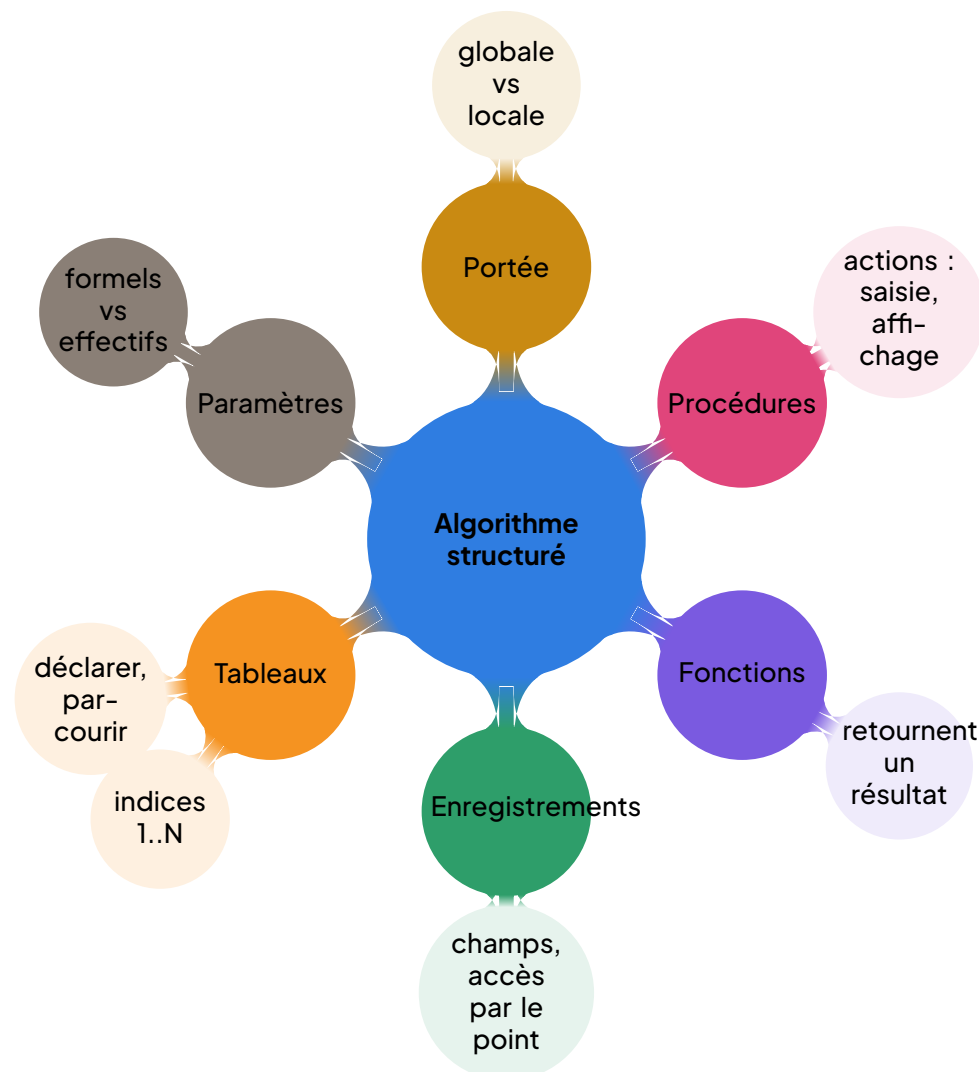
dans `AfficherBulletin`.

- Faire afficher le résultat par la fonction au lieu de le retourner. Une fonction de calcul doit `Retourner` sa valeur ; c'est la procédure d'affichage (ou le programme principal) qui l'écrit à l'écran. Une fonction ne doit pas contenir d'instruction `Ecrire`.
- Modifier un paramètre formel de type simple dans une fonction. Si `E` était un entier passé par valeur, l'assignation `E <- 5` ne modifierait pas l'argument effectif. En pseudo-code Probatoire, on considère que les types simples sont passés par valeur, les structures (tableaux, enregistrements) par référence.

6.6 Critères de qualité d'un algorithme structuré

Deux algorithmes peuvent donner le même résultat, l'un étant excellent et l'autre illisible. Les correcteurs (et tes futurs collègues programmeurs) jugent sur quatre critères :

- **Lisibilité** : indentation régulière (les blocs `Pour`, `Si`, corps de procédures sont décalés de 2 ou 3 espaces), une instruction par ligne ;
- **Noms explicites** : `MoyenneEleve` et `AfficherBulletin` se comprennent sans commentaire ; `F1` et `P2` n'évoquent rien ;
- **Absence de duplication** : le calcul de la moyenne est écrit une seule fois, dans la fonction ; si l'on devait corriger la formule (par exemple ajouter des coefficients), on ne la modifierait qu'à un seul endroit (principe DRY : Don't Repeat Yourself) ;
- **Commentaires utiles** : ils expliquent le rôle d'un bloc (ce qu'il fait et pourquoi), pas ce que la ligne dit déjà d'elle-même.



Carte mentale récapitulative de la leçon : les six grandes notions et leurs points clés.

LESAVAIS-TU ? La programmation structurée, une révolution

L'organisation que tu viens d'apprendre — découper un programme en sous-programmes aux rôles bien définis, avec des variables locales et des paramètres explicites — s'appelle la **programmation structurée**. Théorisée à la fin des années 1960 par des pionniers comme Edsger Dijkstra (auteur du célèbre article « Go To Statement Considered Harmful »), elle a mis fin aux programmes « en spaghetti » où l'on sautait d'un point à un autre sans logique. Ce principe est devenu le fondement des grands langages : Pascal (conçu par Niklaus Wirth précisément pour l'enseignement), C, puis Python et Java. Quand tu écriras tes premiers programmes en C cette année, tu retrouveras exactement la même architecture : déclarations globales, fonctions, puis une fonction principale `main()` très courte.

EXERCICE 8 À toi de structurer

La bibliothèque de ton lycée veut gérer ses 100 livres. Chaque livre est décrit par un titre, un auteur, une année d'édition et un état (disponible ou emprunté). (1) Propose les structures de données adaptées. (2) Identifie les sous-algorithmes nécessaires (saisie du catalogue, affichage des livres disponibles, calcul du nombre de livres empruntés) en précisant pour chacun s'il s'agit d'une fonction ou d'une procédure. (3) Écris l'algorithme complet en respectant l'ordre des déclarations.

CORRIGÉ Exercice : gestion de la bibliothèque

(1) Un enregistrement `Livre` (titre, auteur : chaînes ; année : entier ; emprunte : booléen) et un tableau global `Catalogue` : `Tableau[1..100]` de `Livre`.

(2) `SaisirCatalogue` : procédure (action de saisie, plusieurs effets) ; `AfficherDisponibles` : procédure (action d'affichage) ; `NbEmpruntes` : fonction retournant un entier (un seul résultat calculé).

(3) Ordre du texte : déclaration du type `Livre`, puis des constantes et du tableau global `Catalogue`, puis la fonction `NbEmpruntes` (qui parcourt le tableau et compte les fiches dont le champ `emprunte` est vrai), puis les deux procédures, et enfin l'algorithme principal réduit à trois appels : `SaisirCatalogue()`, `AfficherDisponibles()`, `Ecrire(NbEmpruntes())`.

À RETENIR L'essentiel de la synthèse

- La résolution d'un problème suit 4 étapes : analyser les données, choisir les structures (tableau, enregistrement, tableau d'enregistrements), découper en sous-algorithmes, puis assembler.
- Architecture type : types → constantes et variables globales → fonctions et procédures → algorithme principal court.
- Une tâche qui retourne un seul résultat est une fonction ; une tâche qui agit (saisie, affichage, modification multiple) est une procédure.
- Toute entité (type, constante, variable, sous-algorithme) doit être déclarée avant d'être utilisée.
- La trace d'exécution est l'outil de vérification et la clé des questions de compréhension au Probatoire.
- Un bon algorithme est lisible, utilise des noms explicites, ne duplique aucun code (factorisation) et est commenté avec pertinence.
- Distinguer clairement paramètre formel (déclaration) et paramètre effectif (appel) ; respecter la portée des variables (globale vs locale).

7 ACTIVITÉ D'INTÉGRATION

8 LEÇON 02 : STRUCTURES DE DONNÉES ET SOUS-ALGORITHMES

8.1 Objectifs de l'activité

À l'issue de cette activité d'intégration, tu seras capable de :

- déclarer et utiliser un **type enregistrement** pour regrouper les informations d'un élève ;
- déclarer, remplir et parcourir un **tableau** d'enregistrements ;
- décomposer un problème en **sous-algorithmes** : écrire une **fonction** qui retourne un résultat et des **procédures** qui réalisent des actions ;
- distinguer clairement **variables globales** et **variables locales**, **paramètres formels** et **paramètres effectifs** ;
- effectuer une **trace d'exécution** pour vérifier un algorithme et pratiquer la relecture croisée (chasse aux erreurs de portée et de passage de paramètres).

8.2 Situation

Le Lycée de Nkol-Eton, dans la région du Centre, vient de recevoir un ordinateur pour sa salle informatique. Monsieur Essono, professeur principal de la classe de Première TI 2, en a assez de calculer les moyennes à la main à chaque fin de séquence : les erreurs de report sur les bulletins provoquent des contestations de la part des parents lors des réunions. Il te confie la mission de concevoir l'algorithme d'un petit logiciel de gestion des bulletins.

Voici les données du problème :

- la classe compte **5 élèves** ;
- chaque élève est caractérisé par son **nom** (chaîne de caractères), ses **3 notes** de la séquence (nombres réels sur 20) et sa **moyenne** (nombre réel, calculée par le programme) ;
- la moyenne est la moyenne arithmétique des 3 notes : $\text{Moy} = \frac{\text{note}_1 + \text{note}_2 + \text{note}_3}{3}$;
- le bulletin affiche, pour chaque élève : son nom, ses 3 notes, sa moyenne arrondie au centième et sa **mention** : « Passable » si $\text{Moy} \geq 10$, « Insuffisant » sinon ;
- à la fin, le programme affiche le **major de la classe** (l'élève de plus forte moyenne) ;
- pour tester le programme, on utilisera les notes réelles de la séquence 1, relevées dans le cahier de notes de M. Essono :

Élève	Note 1	Note 2	Note 3
ABENA Marie	12,5	14	9,5
MBARGA Paul	8	10,5	7
NKoulou Amina	15	16,5	17
ETOA Jean	9,5	11	10,5
FOTSO Brice	13	12	14,5

M. Essono exige un programme bien structuré : chaque tâche (saisie, calcul, affichage) doit être confiée à un sous-algorithme distinct, afin que le programme soit facile à relire et à corriger par ses collègues.

8.3 Consignes

Par groupes de 3 élèves, répondez aux questions suivantes dans l'ordre.

1. **Structuration des données.** Déclarez le type enregistrement `Eleve` comportant les champs `Nom` (chaîne), `Notes` (tableau de 3 réels) et `Moy` (réel). Déclarez ensuite le tableau global `Classe` de 5 éléments de type `Eleve`. Justifiez le choix d'un enregistrement plutôt que de trois tableaux séparés.
2. **La fonction de calcul.** Écrivez la fonction `CalculMoyenne` qui reçoit en paramètre un tableau de 3 réels et retourne la moyenne. Précisez le paramètre formel, son type, et le type de la valeur retournée.
3. **La procédure de saisie.** Écrivez la procédure `SaisirEleve` qui reçoit un numéro d'élève i (entier) et remplit les champs `Nom` et `Notes` de `Classe[i]` par lecture au clavier. La variable `Classe` est-elle ici locale ou globale ? Justifiez.
4. **La procédure d'affichage.** Écrivez la procédure `AfficherBulletin` qui reçoit un numéro d'élève i et affiche le nom, les 3 notes, la moyenne et la mention de `Classe[i]`. La moyenne doit être calculée par un appel à `CalculMoyenne` et stockée dans le champ `Moy`.
5. **L'algorithme principal.** Écrivez l'algorithme `Bulletins` qui : saisit les 5 élèves, calcule et affiche les 5 bulletins, puis détermine et affiche le major de la classe (nom et moyenne). Identifiez, à chaque appel, les paramètres effectifs et les paramètres formels correspondants.
6. **Trace d'exécution.** À l'aide du tableau de notes ci-dessus, calculez à la main la moyenne et la mention de chaque élève, puis indiquez ce qu'affiche exactement le programme à la fin (major compris).
7. **Relecture croisée.** Échangez votre production avec un autre groupe. Traquez : les variables utilisées sans déclaration, les confusions local/global, les appels avec un mauvais nombre ou un mauvais type d'arguments, les indices de tableau hors limites (par exemple `Notes[4]`). Rédigez un rapport d'erreurs.

8.4 Ressources à mobiliser

À RETENIR Boîte à outils de la leçon

- **Tableau** : `Tableau NomTab[1..N] : Type`. **Accès à un élément** : `NomTab[i]`, avec $1 \leq i \leq N$.
- **Enregistrement** : `Type NomType = Enregistrement ... Fin Enregistrement`. **Accès à un champ** : `Variable.Champ`.
- **Fonction** : `Fonction NomFonction(param : Type) : TypeRetour ... Retourner valeur`. Elle retourne un résultat utilisable dans une expression.
- **Procédure** : `Procédure NomProc(param : Type) ... Fin Procédure`. Elle réalise une action (saisie, affichage) sans retourner de valeur.
- **Portée** : une variable déclarée dans un sous-algorithme est **locale** (invisible ailleurs); une variable déclarée dans l'algorithme principal est **globale** (visible partout).
- **Paramètres** : le paramètre **formel** est le nom déclaré dans l'en-tête du sous-algorithme; le paramètre **effectif** (argument) est la valeur transmise lors de l'appel. Nombre, ordre et types doivent correspondre.
- **Moyenne de 3 notes** : $\text{Moy} = \frac{n_1 + n_2 + n_3}{3}$.

8.5 Démarche et corrigé commenté**EXERCICE RÉSOLU 8** Bulletin informatisé du Lycée de Nkol-Eton

Énoncé : construire l'algorithme complet de gestion des bulletins décrit dans la situation, avec la fonction `CalculMoyenne`, les procédures `SaisirEleve` et `AfficherBulletin`, et l'affichage du major.

SOLUTION

Étape 1 — Structuration des données (question 1).

On choisit un enregistrement car les informations d'un élève (nom, notes, moyenne) forment une unité logique : avec trois tableaux séparés, on risquerait de mélanger les indices ; avec un enregistrement, `Classe[i]` désigne tout l'élève i .

```
Type Eleve = Enregistrement
  Nom   : Chaîne
  Notes : Tableau[1..3] de Reel
  Moy   : Reel
Fin Enregistrement

Variable Classe : Tableau[1..5] de Eleve // variable globale
```

Étape 2 — La fonction de calcul (question 2).

Le paramètre formel T est un tableau de 3 réels ; la fonction retourne un réel. La variable s est locale : elle n'existe que pendant l'appel.

```

Fonction CalculMoyenne( $T$  : Tableau[1..3] de Reel) : Reel
Variable  $s$  : Reel
Debut
     $s \leftarrow T[1] + T[2] + T[3]$ 
    Retourner  $s / 3$ 
Fin

```

Étape 3 — La procédure de saisie (question 3).

```

Procédure SaisirEleve( $i$  : Entier)
Variable  $j$  : Entier
Debut
    Ecrire("Nom de l'eleve ",  $i$ , " : ")
    Lire(Classe[ $i$ ].Nom)
    Pour  $j$  de 1 à 3 Faire
        Ecrire("Note ",  $j$ , " : ")
        Lire(Classe[ $i$ ].Notes[ $j$ ])
    Fin Pour
Fin

```

Classe est une **variable globale** : la procédure la modifie directement, c'est justement son rôle. En revanche i (paramètre formel) et j sont locaux à la procédure.

Étape 4 — La procédure d'affichage (question 4).

```

Procédure AfficherBulletin( $i$  : Entier)
Debut
    Classe[ $i$ ].Moy  $\leftarrow$  CalculMoyenne(Classe[ $i$ ].Notes)
    Ecrire("Eleve : ", Classe[ $i$ ].Nom)
    Ecrire("Notes : ", Classe[ $i$ ].Notes[1], " ; ",
        Classe[ $i$ ].Notes[2], " ; ", Classe[ $i$ ].Notes[3])
    Ecrire("Moyenne : ", Classe[ $i$ ].Moy)
    Si Classe[ $i$ ].Moy  $\geq$  10 Alors
        Ecrire("Mention : Passable")
    Sinon
        Ecrire("Mention : Insuffisant")
    Fin Si
Fin

```

On remarque l'appel de fonction dans une affectation : le paramètre effectif `Classe[ $i$ ].Notes` correspond au paramètre formel T (même type : tableau de 3 réels).

Étape 5 — L'algorithme principal (question 5).

```

Algorithme Bulletins
Variable  $i$ , indiceMajor : Entier
Variable maxMoy : Reel
Debut
    // Saisie des 5 eleves

```

```

    Pour i de 1 à 5 Faire
        SaisirEleve(i)
    Fin Pour

    // Calcul et affichage des bulletins
    Pour i de 1 à 5 Faire
        AfficherBulletin(i)
    Fin Pour

    // Recherche du major
    indiceMajor ← -1
    maxMoy ← Classe[1].Moy
    Pour i de 2 à 5 Faire
        Si Classe[i].Moy > maxMoy Alors
            maxMoy ← Classe[i].Moy
            indiceMajor ← i
        Fin Si
    Fin Pour

    Ecrire("Major de la classe : ", Classe[indiceMajor].Nom,
        " avec ", maxMoy)
Fin

```

Ici, les variables `i`, `indiceMajor` et `maxMoy` sont locales à l'algorithme principal; seul le tableau `Classe` est global, ce qui permet aux procédures d'y accéder. À l'appel `SaisirEleve(i)`, le paramètre effectif `i` (valeur 1, puis 2, ..., 5) est transmis au paramètre formel `i` de la procédure.

Étape 6 — Trace d'exécution (question 6).

$$\begin{aligned}
 \text{ABENA Marie} &: \frac{12,5 + 14 + 9,5}{3} = \frac{36}{3} = 12,00 \quad \text{Passable} \\
 \text{MBARGA Paul} &: \frac{8 + 10,5 + 7}{3} = \frac{25,5}{3} = 8,50 \quad \text{Insuffisant} \\
 \text{NKoulou Amina} &: \frac{15 + 16,5 + 17}{3} = \frac{48,5}{3} \approx 16,17 \quad \text{Passable} \\
 \text{ETOA Jean} &: \frac{9,5 + 11 + 10,5}{3} = \frac{31}{3} \approx 10,33 \quad \text{Passable} \\
 \text{FOTSO Brice} &: \frac{13 + 12 + 14,5}{3} = \frac{39,5}{3} \approx 13,17 \quad \text{Passable}
 \end{aligned}$$

Le programme affiche donc à la fin : « Major de la classe : NKoulou Amina avec 16,17 ».

Étape 7 — Erreurs typiques à traquer en relecture croisée (question 7).

- utiliser `Notes[j]` au lieu de `Classe[i].Notes[j]` dans la procédure : `Notes` seul n'existe pas, c'est un champ;
- écrire `Pour j de 1 à 4` : l'indice 4 est hors du tableau `Notes[1..3]`;
- appeler `CalculMoyenne(Classe[i])` au lieu de `CalculMoyenne(Classe[i].Notes)` : le

type transmis (enregistrement) ne correspond pas au paramètre formel (tableau de réels);

- déclarer `Moy` comme variable locale dans `AfficherBulletin` sans jamais la ranger dans `Classe[i].Moy` : la recherche du major échoue alors, car les moyennes n'ont pas été mémorisées;
- oublier d'initialiser `maxMoy` avant la boucle de recherche du major.

ATTENTION Pièges révélés par l'activité

- Une fonction retourne une valeur : on l'utilise dans une affectation ou une expression. Une procédure fait quelque chose : on l'appelle seule, sur sa propre ligne. Ne jamais écrire `x <- AfficherBulletin(i)`.
- Le passage de paramètre exige une correspondance stricte : même nombre, même ordre, mêmes types entre arguments effectifs et paramètres formels.
- Modifier une variable globale dans une procédure est possible, mais il faut le faire consciemment et le documenter : c'est une source fréquente d'effets de bord difficiles à déboguer.

8.6 Bilan de l'activité

Cette activité a permis de mobiliser simultanément tous les savoir-faire de la leçon dans un problème authentique de la vie scolaire camerounaise :

- le **tableau** et l'**enregistrement** se sont révélés complémentaires : l'enregistrement regroupe les données hétérogènes d'un élève, le tableau permet de traiter les 5 élèves par une simple boucle;
- la décomposition en **sous-algorithmes** a rendu le programme lisible : chaque tâche (saisir, calculer, afficher, trouver le major) est isolée, testable et réutilisable;
- la distinction **fonction/procédure** s'est éclairée par la pratique : `CalculMoyenne` retourne un résultat réutilisable, tandis que `SaisirEleve` et `AfficherBulletin` réalisent des actions;
- la **trace d'exécution** a validé l'algorithme sur des données réelles avant toute programmation, et la relecture croisée a montré que la plupart des erreurs viennent de la **portée des variables** et du **passage des paramètres**;
- enfin, la recherche du major a illustré un algorithme classique de parcours de tableau (recherche du maximum), qui sera approfondi dans les leçons sur le tri et la recherche.

Tu es maintenant prêt à transposer cet algorithme en langage C ou à l'enrichir : ajouter le rang de chaque élève, le nombre de mentions, ou la moyenne générale de la classe ne demande que quelques lignes supplémentaires, grâce à la bonne structuration du programme.

9 MÉTHODES ET EXERCICES RÉSOLUS

10 STRUCTURES DE DONNÉES ET SOUS-ALGORITHMES

10.1 Structures de données : panorama et tableaux

Fiche méthode 1 : Énumérer et choisir une structure de données

MÉTHODE Choisir la bonne structure de données

Pour répondre à une question du type « quelle structure de données utiliser ? », procède ainsi :

1. Compte les valeurs à mémoriser : une seule valeur → variable simple ; plusieurs valeurs de même type → **tableau** ; plusieurs valeurs de types différents décrivant un même objet → **enregistrement**.
2. Vérifie l'homogénéité : dans un tableau, tous les éléments ont le même type (que des entiers, que des réels...). Si tu mélanges un nom (chaîne), un âge (entier) et une moyenne (réel), c'est un enregistrement.
3. Identifie le mode d'accès : tableau → accès par indice ($T[i]$) ; enregistrement → accès par champ ($E.nom$).
4. Justifie ta réponse en une phrase : « les données sont homogènes et nombreuses, donc un tableau convient ».

Réflexe Bac : cite toujours au moins trois structures de données quand on te demande d'en énumérer : la variable simple, le tableau, l'enregistrement (et tu peux ajouter la chaîne de caractères).

EXERCICE RÉSOLU 9 Choisir une structure pour un lycée de Yaoundé

Le proviseur d'un lycée de Yaoundé veut mémoriser : (a) les 25 notes d'informatique d'une classe de Première TI ; (b) pour chaque élève, son nom, sa date de naissance et sa moyenne générale. Pour chaque cas, indique la structure de données adaptée en justifiant.

SOLUTION

Raisonnement. On applique la fiche méthode : on compte les valeurs et on teste l'homogénéité.

Cas (a) : les 25 notes. Il s'agit de 25 valeurs de même type (des réels, car une note peut va-

loir 12,5). Elles sont homogènes et nombreuses : on utilise donc un **tableau** à une dimension de 25 réels.

Variable
Notes : Tableau[1..25] de Reel

Cas (b) : nom + date de naissance + moyenne. Ces trois informations décrivent un même objet (un élève) mais sont de types différents : chaîne de caractères, date (ou chaîne), réel. Un tableau est impossible car il exige l'homogénéité. On utilise donc un **enregistrement**.

Type
Eleve = Enregistrement
nom : Chaîne
dateNaissance : Chaîne
moyenne : Reel
Fin Enregistrement

Conclusion. Les 25 notes, homogènes, sont stockées dans un tableau de 25 réels ; les informations hétérogènes d'un élève sont regroupées dans un enregistrement à trois champs. Si l'on voulait gérer toute la classe, on combinerait les deux : un tableau d'enregistrements de type `Eleve`.

Fiche méthode 2 : Déclarer et manipuler un tableau à une dimension

MÉTHODE Déclarer, initialiser, parcourir, afficher, modifier un tableau

1. Déclarer en précisant le nom, la plage d'indices et le type des éléments : `T : Tableau[1..N] de Type`.
2. Initialiser : ne jamais utiliser un tableau non rempli. Deux possibilités : saisie élément par élément dans une boucle, ou affectation directe (`T[1] ← 12`).
3. Accéder à un élément par son indice : `T[i]`. L'indice i doit vérifier $1 \leq i \leq N$.
4. Parcourir systématiquement avec une boucle `Pour` dont le compteur va de 1 à N : c'est le schéma universel pour afficher, sommer, compter ou rechercher.
5. Modifier un élément par affectation : `T[i] ← nouvelleValeur`.

Formule à retenir : le nombre d'éléments d'un tableau `Tableau[1..N]` est exactement N . Le premier indice est 1, le dernier est N .

EXERCICE RÉSOLU 10 Moyenne des notes d'une classe

Écris un algorithme qui : (1) déclare un tableau `Notes` de 10 réels ; (2) fait saisir les 10 notes par l'utilisateur ; (3) calcule et affiche la moyenne de la classe ; (4) affiche ensuite toutes les notes supérieures ou égales à 10.

SOLUTION

Raisonnement. On applique le schéma de parcours deux fois : une première boucle pour la saisie, un calcul de somme, puis une seconde boucle avec un test pour l'affichage sélectif.

```
Algorithme MoyenneClasse
Variable
    Notes : Tableau[1..10] de Reel
    i : Entier
    somme, moyenne : Reel
Debut
    // (2) Saisie et initialisation du tableau
    Pour i de 1 a 10 Faire
        Ecrire("Entrez la note numero ", i, " : ")
        Lire(Notes[i])
    Fin Pour

    // (3) Calcul de la moyenne
    somme <- 0
    Pour i de 1 a 10 Faire
        somme <- somme + Notes[i]
    Fin Pour
    moyenne <- somme / 10
    Ecrire("La moyenne de la classe est : ", moyenne)

    // (4) Affichage des notes >= 10
    Ecrire("Notes superieures ou egales a 10 :")
    Pour i de 1 a 10 Faire
        Si Notes[i] >= 10 Alors
            Ecrire(Notes[i])
        Fin Si
    Fin Pour
Fin
```

Justifications. L'accumulateur `somme` est obligatoirement initialisé à 0 avant la boucle, sinon il contient une valeur indéterminée. La moyenne est bien somme / N avec $N = 10$ éléments (indices de 1 à 10). Le test `Notes[i] >= 10` est placé à l'intérieur de la boucle pour examiner chaque élément.

Conclusion. L'algorithme saisit les 10 notes, affiche la moyenne $\frac{\text{somme}}{10}$, puis liste les notes admises (≥ 10), grâce à deux parcours complets du tableau.

10.2 Les enregistrements

Fiche méthode 3 : Déclarer et utiliser un enregistrement

MÉTHODE Manipuler un enregistrement

1. Définir le type enregistrement dans la partie Type, en listant les champs avec leurs types : `Produit = Enregistrement ... Fin Enregistrement.`
2. Déclarer une variable de ce type : `p : Produit.`
3. Initialiser chaque champ par affectation ou par saisie, avec la notation pointée : `p.nom ← "Savon".`
4. Accéder à un champ en lecture : `Ecrire(p.prix).`
5. Modifier un champ comme une variable ordinaire : `p.prix ← p.prix * 1,1` (en pseudo-code, le séparateur décimal est la virgule).
6. Pour gérer plusieurs objets, déclarer un tableau d'enregistrements : `stock : Tableau[1..50] de Produit;` on accède alors à un champ par `stock[i].prix.`

Réflexe : le point « . » est l'opérateur d'accès au champ. `variable.champ` se manipule exactement comme une variable du type du champ.

EXERCICE RÉSOLU 11 Gestion d'une boutique à Douala

Une boutique de Bonanjo vend des articles caractérisés par un nom (chaîne), un prix unitaire en FCFA (entier) et une quantité en stock (entier). (a) Définis le type enregistrement `Article`. (b) Écris un algorithme qui saisit un article, puis affiche la valeur totale de son stock (`prix × quantité`).

SOLUTION

(a) Définition du type. On regroupe les trois champs hétérogènes dans un enregistrement (déclaration globale, avant l'algorithme principal) :

```
Type
  Article = Enregistrement
    nom : Chaîne
    prix : Entier
    quantite : Entier
  Fin Enregistrement
```

(b) Algorithme. On déclare une variable de type `Article`, on initialise ses champs par saisie, puis on calcule la valeur du stock en accédant aux champs en lecture.

```
Algorithme ValeurStock
```

```

Variable
  a : Article
  valeur : Entier
Debut
  Ecrire("Nom de l'article : ")
  Lire(a.nom)
  Ecrire("Prix unitaire (FCFA) : ")
  Lire(a.prix)
  Ecrire("Quantite en stock : ")
  Lire(a.quantite)

  valeur ← a.prix * a.quantite
  Ecrire("La valeur du stock de ", a.nom, " est : ", valeur, " FCFA")
Fin

```

Justifications. Chaque champ est initialisé avant d'être utilisé (sinon le calcul porterait sur des valeurs indéterminées). Le produit `a.prix * a.quantite` est licite car les deux champs sont de type `Entier`; le résultat est rangé dans `valeur`, également entière. Exemple numérique : pour un article à 500 FCFA en 30 exemplaires, `valeur = 500 × 30 = 15 000 FCFA`. **Conclusion.** L'enregistrement `Article` regroupe les trois informations hétérogènes, et la notation pointée `a.prix`, `a.quantite` permet le calcul de la valeur totale du stock.

10.3 Sous-algorithmes : rôle et fonctions

Fiche méthode 4 : Décrire le rôle d'un sous-algorithme et écrire une fonction

MÉTHODE Concevoir et appeler une fonction

1. Identifier le besoin : un sous-algorithme sert à découper un problème complexe en tâches simples, à éviter la duplication de code et à réutiliser un traitement.
2. Choisir entre fonction et procédure : si le traitement doit retourner un résultat unique (un calcul), c'est une **fonction**; s'il effectue des actions sans résultat unique (affichage, saisie, modification), c'est une **procédure**.
3. Déclarer la fonction : nom, paramètres formels typés, type du résultat, variables locales, corps terminé par `Retourner(resultat)`.
4. Appeler la fonction dans l'algorithme principal en fournissant des arguments (paramètres effectifs) en nombre, ordre et type compatibles : `m ← Maximum(a, b)`.
5. Exploiter le résultat : la valeur retournée est affectée à une variable ou utilisée dans une expression.

Réflexe Bac : une fonction possède toujours un type de retour et au moins une instruction `Retourner`.

EXERCICE RÉSOLU 12**Fonction de calcul de la moyenne de deux notes**

(a) Explique le rôle d'un sous-algorithme dans un algorithme. (b) Écris une fonction `Moyenne` qui reçoit deux réels et retourne leur moyenne. (c) Écris l'algorithme principal qui saisit les notes d'un élève en Maths et en Informatique, appelle la fonction et affiche la moyenne.

SOLUTION

(a) Rôle d'un sous-algorithme. Un sous-algorithme est un bloc d'instructions nommé qui réalise une tâche précise. Il permet de : décomposer un problème complexe en sous-problèmes simples ; écrire une seule fois un traitement utilisé plusieurs fois (réutilisabilité) ; rendre l'algorithme principal plus lisible et plus facile à tester et à corriger.

(b) La fonction. Deux paramètres formels réels, un résultat de type réel (déclarée avant l'algorithme principal) :

```
Fonction Moyenne(n1 : Reel, n2 : Reel) : Reel
Variable
  m : Reel
Debut
  m <- (n1 + n2) / 2
  Retourner(m)
Fin
```

(c) Algorithme principal. On appelle la fonction avec des arguments effectifs (les variables saisies) :

```
Algorithme CalculMoyenne
Variable
  noteMaths, noteInfo, moy : Reel
Debut
  Ecrire("Note de Maths : ")
  Lire(noteMaths)
  Ecrire("Note d'Informatique : ")
  Lire(noteInfo)
  moy <- Moyenne(noteMaths, noteInfo)
  Ecrire("La moyenne est : ", moy)
Fin
```

Justifications. Les paramètres formels `n1` et `n2` reçoivent, lors de l'appel, les valeurs des arguments effectifs `noteMaths` et `noteInfo` : correspondance par position, avec le même type (réel). La variable `m` est locale à la fonction : elle n'existe pas dans l'algorithme principal. Exemple : pour 14 et 16, la fonction retourne $(14 + 16) / 2 = 15$.

Conclusion. La fonction `Moyenne` encapsule le calcul ; l'algorithme principal se contente de saisir les données, d'appeler la fonction et d'afficher le résultat retourné.

10.4 Les procédures

Fiche méthode 5 : Écrire et appeler une procédure

MÉTHODE Concevoir une procédure

1. Constaté qu'aucun résultat unique n'est attendu : la procédure fait quelque chose (afficher, saisir, permuter) mais ne retourne rien : pas de type de retour, pas de Retourner.
2. Déclarer avec le mot-clé `Procédure`, suivi du nom et des paramètres formels typés entre parenthèses.
3. Écrire le corps : instructions d'affichage, de saisie ou de traitement, en utilisant les paramètres et les variables locales.
4. Appeler la procédure comme une instruction autonome : `AfficherEntete("Lycee de Bafoussam")` — jamais dans une affectation.

Différence capitale : fonction $\Rightarrow$ retourne une valeur (utilisable dans une expression);
procédure $\Rightarrow$ exécute des actions (appelée seule, sur une ligne).

EXERCICE RÉSOLU 13 Procédure d'affichage d'un bulletin

Écris une procédure `AfficherMention` qui reçoit une moyenne (réel) et affiche la mention correspondante : « Ajourné » si moyenne < 10 , « Passable » si $10 \leq \text{moyenne} < 12$, « Assez bien » si $12 \leq \text{moyenne} < 14$, « Bien » sinon. Puis écris l'algorithme principal qui saisit la moyenne d'un élève et appelle la procédure.

SOLUTION

Raisonnement. Le traitement demandé est un affichage : aucun résultat n'est à retourner, donc on choisit une procédure. Les cas sont disjoints et couvrent tout $\mathbb{R}$: on utilise des alternatives imbriquées avec des conditions soigneusement bornées.

```
Procédure AfficherMention(moy : Reel)
Debut
  Si moy < 10 Alors
    Ecrire("Mention : Ajourné")
  Sinon
    Si moy < 12 Alors
      Ecrire("Mention : Passable")
    Sinon
      Si moy < 14 Alors
        Ecrire("Mention : Assez bien")
      Sinon
        Ecrire("Mention : Bien")
```

Fin Si
Fin Si
Fin Si
Fin
Algorithme Bulletin
Variable
moyenne : Reel
Debut
Ecrire("Entrez la moyenne de l'eleve : ")
Lire(moyenne)
AfficherMention(moyenne)
Fin

Justifications. Les tests successifs $\text{moy} < 10, \text{moy} < 12, \text{moy} < 14$ dans des *Sinon* imbriqués réalisent exactement les intervalles demandés : par exemple, atteindre le deuxième test signifie déjà que $\text{moy} \geq 10$, donc $\text{moy} < 12$ équivaut à $10 \leq \text{moy} < 12$. Le dernier *Sinon* correspond à $\text{moy} \geq 14$. Vérification : 9,5 → Ajourné ; 11 → Passable ; 13 → Assez bien ; 15 → Bien. L'appel `AfficherMention(moyenne)` est une instruction seule, sans affectation, car la procédure ne retourne rien.

Conclusion. La procédure `AfficherMention` réalise l'affichage de la mention par alternatives imbriquées ; l'algorithme principal saisit la moyenne puis l'appelle avec l'argument effectif `moyenne`.

10.5 Variables globales, variables locales, paramètres

Fiche méthode 6 : Distinguer variables globales, variables locales, paramètres formels et effectifs

MÉTHODE Analyser la portée des variables et le passage des paramètres

1. Repérer le lieu de déclaration : une variable déclarée dans l'algorithme principal (avant les sous-algorithmes, ou dans la partie commune) est **globale** : visible partout. Une variable déclarée à l'intérieur d'une fonction ou procédure est **locale** : elle n'existe que pendant l'exécution de ce sous-algorithme.
2. Distinguer paramètre formel et effectif : le **paramètre formel** est le nom déclaré dans l'en-tête du sous-algorithme (`n` dans `Fonction Carre(n : Reel)`) ; le **paramètre effectif** (ou argument) est la valeur ou variable fournie lors de l'appel (`x` dans `Carre(x)`).
3. Vérifier la correspondance : les arguments doivent correspondre aux paramètres formels en nombre, en ordre et en type.
4. Conclure sur la portée : modifier une variable locale ne change rien à l'extérieur ; une variable globale modifiée dans un sous-algorithme est modifiée pour tout l'algorithme (à utiliser avec prudence).

Réflexe Bac : dans un tableau « variable / globale ou locale / rôle », le lieu de déclaration est le seul critère.

EXERCICE RÉSOLU 14 Portée des variables dans un calcul de surface

On considère l'algorithme suivant (le sous-algorithme est placé avant le principal par convention) :

```

Fonction Surface(L : Reel, l : Reel) : Reel
Variable
  s : Reel
Debut
  s <- L * l
  Retourner(s)
Fin

Algorithme Terrain
Variable
  longueur, largeur, surf : Reel // variables de l'algorithme principal
Debut
  Ecrire("Longueur (m) : ")
  Lire(longueur)
  Ecrire("Largeur (m) : ")
  Lire(largeur)
  surf <- Surface(longueur, largeur)
  Ecrire("Surface : ", surf, " m2")
Fin
  
```

(1) Pour chaque variable (longueur, largeur, surf, L, l, s), précisez s'il s'agit d'une variable globale, d'une variable locale ou d'un paramètre formel. (2) Dans l'appel Surface(longueur, largeur), identifie les paramètres effectifs. (3) Que vaut surf si l'utilisateur saisit 20 et 12 ?

SOLUTION

(1) Nature de chaque variable. On applique le critère du lieu de déclaration :

Nom	Nature	Justification
longueur	Variable globale	Déclarée dans l'algorithme principal
largeur	Variable globale	Déclarée dans l'algorithme principal
surf	Variable globale	Déclarée dans l'algorithme principal
L, l	Paramètres formels	Déclarés dans l'en-tête de la fonction
s	Variable locale	Déclarée à l'intérieur de la fonction Surface

La variable `s` n'existe que pendant l'exécution de `Surface` : elle est détruite dès que la fonction se termine, et l'algorithme principal ne peut pas y accéder.

(2) Paramètres effectifs. Dans l'appel `Surface(longueur, largeur)`, les paramètres effectifs (arguments) sont `longueur` et `largeur`. La correspondance se fait par position : la valeur de `longueur` est copiée dans `L`, celle de `largeur` dans `l`. Les types correspondent (réels partout) : l'appel est correct.

(3) Calcul. Pour `longueur = 20` et `largeur = 12` :

$$s = L \times l = 20 \times 12 = 240.$$

La fonction retourne 240, qui est affecté à `surf`. Le programme affiche donc « Surface : 240 m² ».

Conclusion. `longueur`, `largeur` et `surf` sont globales ; `s` est locale à la fonction ; `L` et `l` sont les paramètres formels recevant les arguments effectifs `longueur` et `largeur` ; la surface affichée vaut 240 m².

10.6 Synthèse : structurer un algorithme complet

Fiche méthode 7 : Structurer un algorithme complet avec sous-algorithmes

MÉTHODE Organiser un algorithme complet

1. Décomposer l'énoncé en tâches : saisie, traitements (calculs), affichages. Chaque tâche répétée ou autonome devient un sous-algorithme.
2. Décider fonction ou procédure pour chaque tâche : résultat unique → fonction ; action sans résultat → procédure.
3. Choisir les structures de données : tableau pour les séries homogènes, enregistrement pour les fiches hétérogènes.
4. Rédiger dans l'ordre : définitions de types (enregistrements), puis fonctions et procédures, puis l'algorithme principal qui appelle les sous-algorithmes.
5. Tester mentalement avec un jeu de valeurs simple et vérifier chaque affichage.

EXERCICE RÉSOLU 15 Statistiques des notes d'une classe de Première TI

Écris un algorithme complet qui : saisit les notes d'informatique de $N = 5$ élèves dans un tableau (procédure `Saisir`) ; calcule la moyenne de la classe (fonction `MoyenneTab`) ; affiche la moyenne et le nombre d'élèves ayant une note supérieure ou égale à la moyenne (dans l'algorithme principal).

SOLUTION

Raisonnement. Décomposition : la saisie est une action → procédure ; la moyenne est un résultat unique → fonction ; le comptage se fait dans le programme principal par parcours du tableau. La structure de données est un tableau de réels (données homogènes).

```

Procédure Saisir(T : Tableau[1..5] de Reel)
Variable
  i : Entier
Debut
  Pour i de 1 à 5 Faire
    Ecrire("Note de l'élève ", i, " : ")
    Lire(T[i])
  Fin Pour
Fin

Fonction MoyenneTab(T : Tableau[1..5] de Reel) : Reel
Variable
  i : Entier
  somme : Reel
Debut
  somme ← 0
  Pour i de 1 à 5 Faire
    somme ← somme + T[i]
  Fin Pour
  Retourner(somme / 5)
Fin

Algorithme Statistiques
Variable
  Notes : Tableau[1..5] de Reel
  i, compteur : Entier
  moy : Reel
Debut
  Saisir(Notes)
  moy ← MoyenneTab(Notes)
  Ecrire("Moyenne de la classe : ", moy)

  compteur ← 0
  Pour i de 1 à 5 Faire
    Si Notes[i] >= moy Alors
      compteur ← compteur + 1
    Fin Si
  Fin Pour
  Ecrire(compteur, " élève(s) ont une note >= à la moyenne")
Fin

```

Justifications. La procédure `Saisir` remplit le tableau passé en paramètre : pas de valeur à retourner, c'est bien une procédure. La fonction `MoyenneTab` retourne un réel : $\text{moyenne} = \frac{\text{somme}}{5}$. L'accumulateur `compteur` est initialisé à 0 puis incrémenté de 1 à chaque note vérifiant $\text{Notes}[i] \geq \text{moy}$.

Test mental. Avec les notes 8, 12, 10, 14, 6 : somme = $8 + 12 + 10 + 14 + 6 = 50$, donc moy = $50/5 = 10$. Les notes ≥ 10 sont 12, 10 et 14 : le programme affiche « Moyenne de la classe : 10 » puis « 3 eleve(s) ont une note $\geq$ a la moyenne ». C'est cohérent.

Conclusion. L'algorithme est structuré en trois blocs : une procédure de saisie, une fonction de calcul de moyenne, et un programme principal qui orchestre les appels puis compte les notes au moins égales à la moyenne.

ATTENTION Les 5 erreurs qui coûtent des points au Bac

1. **Indice hors limites.** Écrire `Pour i de 0 a N` ou accéder à `T[N+1]` sur un tableau déclaré `Tableau[1..N]` est une erreur classique : les indices valides vont de 1 à N exactement. Vérifie toujours les bornes de ta boucle.
2. **Accumulateur non initialisé.** Oublier `somme ← 0` ou `compteur ← 0` avant la boucle fausse tous les calculs de moyenne et de comptage : la variable contient alors une valeur indéterminée. Toute variable de cumul doit être initialisée avant la boucle, jamais dedans.
3. **Confondre fonction et procédure.** Déclarer une fonction sans `Retourner`, ou écrire `x ← AfficherMenu()` alors qu'`AfficherMenu` est une procédure, sont des fautes de conception : une fonction retourne une valeur typée, une procédure s'appelle seule, sans affectation.
4. **Mélanger paramètres formels et effectifs.** Dans l'en-tête on écrit les paramètres formels avec leurs types (`Fonction Max(a : Reel, b : Reel) : Reel`) ; lors de l'appel on écrit les arguments sans type (`Max(x, y)`). Inverser les deux, ou appeler avec un nombre ou un ordre d'arguments incorrect, est sanctionné.
5. **Croire qu'une variable locale est accessible partout.** Utiliser dans l'algorithme principal une variable déclarée dans une fonction (ou inversement, modifier une variable globale sans s'en rendre compte) est une erreur de portée : une variable locale n'existe que dans son sous-algorithme. Au Bac, justifie toujours « globale » ou « locale » par le lieu de déclaration.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 9 QCM : Structures de données et sous-algorithmes

Pour chaque question, une seule réponse est correcte. Recopiez le numéro et la lettre choisie.

1. Quelle instruction permet de déclarer un tableau `T` de 20 entiers en pseudo-code (style Algobox/Français)?
 - A. Tableau `T[20]` en Entier
 - B. `T <- Tableau(20)`
 - C. `Dim T(20) Entier`
 - D. `Var T : Tableau[1..20] d'entiers`
2. Un enregistrement (structure) permet de :
 - A. Stocker plusieurs valeurs du même type sous un même nom.
 - B. Regrouper des données de types différents liées à une même entité.
 - C. Répéter une suite d'instructions un nombre déterminé de fois.
 - D. Définir une variable accessible uniquement dans le programme principal.
3. La différence fondamentale entre une fonction et une procédure est que :
 - A. La fonction retourne une valeur (via `Retourner`), la procédure n'en retourne pas.
 - B. La procédure accepte des paramètres, la fonction n'en accepte pas.
 - C. La fonction ne peut pas modifier ses paramètres, la procédure le peut.
 - D. La procédure s'appelle avec `Appeler`, la fonction s'appelle par son nom dans une expression.
4. Soit l'appel `Calculer(5, x)` où `x` est une variable du programme principal. Dans l'entête `Procédure Calculer(a : Entier, b : Entier)`, `a` et `b` sont appelés :
 - A. Paramètres effectifs (ou arguments).
 - B. Paramètres formels.
 - C. Variables locales.
 - D. Variables globales.

EXERCICE 10 Vrai ou Faux ? Justifiez

Indiquez si chaque affirmation est Vraie ou Fausse et justifiez brièvement votre réponse.

1. L'indice du premier élément d'un tableau déclaré `Tableau T[1..N]` est toujours 0.
2. Une variable déclarée à l'intérieur d'une fonction est une variable globale au programme principal.
3. Un paramètre formel passé par valeur ne peut pas être modifié par le sous-algorithme pour affecter l'argument d'appel.
4. Une procédure peut être appelée dans une expression arithmétique (ex : `x <- 5 + MaProc(3)`).
5. Le champ `Prix` d'un enregistrement `Produit` déclaré `Var P : Produit` s'accède par `P.Prix` ou `P->Prix`.

EXERCICE 11 Définitions et vocabulaire

Donnez une définition précise et concise pour chacun des termes suivants, en donnant un exemple de déclaration en pseudo-code si pertinent :

1. Enregistrement (ou Structure).
2. Variable locale vs Variable globale (portée).
3. Paramètre formel vs Paramètre effectif (argument).
4. Effet de bord (lié aux procédures et variables globales).

EXERCICE 12 Lecture et exécution manuelle (traçage)

On donne l'algorithme suivant :

```

Variables
  a, b, c : Entier
  T : Tableau[1..5] d'entiers

Debut
  a <- 10
  b <- 3
  T[1] <- 4; T[2] <- 7; T[3] <- 2; T[4] <- 9; T[5] <- 5
  c <- SommePremiers(T, 3) // Appel de la fonction ci-dessous
  Afficher "c = ", c
Fin

Fonction SommePremiers(Tab : Tableau[1..*] d'entiers, n : Entier) : Entier

```

```

Var i, s : Entier
Debut
  s <- 0
  Pour i de 1 à n faire
    s <- s + Tab[i]
  FinPour
  Retourner s
Fin

```

1. Quelle est la valeur affichée pour `c` à la fin de l'exécution ?
2. Que se passe-t-il si on appelle `SommePremiers(T, 6)` ? Justifiez.
3. Les variables `i` et `s` de la fonction sont-elles locales ou globales ? Pourquoi ?

11.2 Je m'entraîne

EXERCICE 13 Manipulation de tableau à une dimension

Écrire un algorithme complet qui :

1. Demande à l'utilisateur la taille `N` du tableau (entier entre 1 et 50).
2. Déclare un tableau `Notes` de `N` réels.
3. Remplit le tableau par saisie au clavier (boucle `Pour`).
4. Calcule et affiche la plus grande note (maximum) et sa position (indice) dans le tableau.
5. Calcule et affiche le nombre de notes supérieures ou égales à 10 (moyenne).

Utilisez une fonction `Maximum(T, N)` qui renvoie la valeur du maximum, et une procédure `SaisieTableau(T, N)` pour le remplissage.

EXERCICE 14 Enregistrement et tableau d'enregistrements

On souhaite gérer les élèves d'une classe. Un élève est caractérisé par : son `Matricule` (chaîne), son `Nom` (chaîne), sa `Moyenne` (réel).

1. Écrire la déclaration du type `Eleve` (enregistrement) et la déclaration d'un tableau `Classe` pouvant contenir jusqu'à 40 élèves.
2. Écrire une procédure `SaisirEleve(var E : Eleve)` qui saisit les informations d'un élève (passage par référence pour modifier l'enregistrement).
3. Écrire une fonction `MeilleurEleve(Classe : Tableau[1..N] d'Eleve, N : Entier) :`

Eleve qui retourne l'enregistrement de l'élève ayant la meilleure moyenne.

4. Écrire l'algorithme principal qui saisit N (nombre d'élèves), remplit le tableau `Classe` en appelant `SaisirEleve`, puis affiche le nom et la moyenne du meilleur élève en appelant `MeilleurEleve`.

EXERCICE 15 Fonction récursive vs itérative : Factorielle

1. Écrire une fonction itérative `FactIter(N : Entier) : Entier` qui calcule $N!$ (factorielle). Gérer le cas $N < 0$ (retourner -1 ou afficher erreur).
2. Écrire une fonction récursive `FactRec(N : Entier) : Entier` qui calcule $N!$.
3. Écrire un algorithme principal qui demande N à l'utilisateur, appelle les deux fonctions et affiche les résultats pour vérifier qu'ils sont identiques.
4. Question de cours : Quel est l'avantage principal de la version itérative sur la version récursive en termes de mémoire (pile d'exécution) ?

EXERCICE 16 Portée des variables et passage de paramètres

On donne le squelette d'algorithme suivant. Complétez les parties manquantes (déclarations, corps) pour qu'il soit syntaxiquement correct et fonctionnel. Identifiez ensuite les variables globales, locales, paramètres formels et effectifs.

```

Variables
    // ... Déclarations globales ...

Debut
    // ... Initialisation ...
    Appeler Modifier(X, Y) // Appel 1
    Afficher "Après Appel 1 : X=", X, " Y=", Y
    Z ← Calculer(X, Y) // Appel 2 (Fonction)
    Afficher "Z = ", Z
Fin

Procédure Modifier(var A : Entier, B : Entier)
    // ... Corps : A prend la valeur 20, B prend la valeur 30 ...
Fin

Fonction Calculer(P, Q : Entier) : Entier
    Var Locale : Entier
    // ... Corps : Locale ← P + Q ; Retourner Locale ...
Fin
  
```

1. Complétez l'algorithme ci-dessus.
2. Après l'exécution de `Appeler Modifier(X, Y)`, quelles sont les valeurs de `X` et `Y`? Justifiez par le mode de passage (valeur/référence).
3. Listez explicitement : les variables globales, les variables locales à `Modifier`, les variables locales à `Calculer`, les paramètres formels de `Modifier`, les paramètres effectifs de l'appel 1.

11.3 Je me prépare au Probatoire

EXERCICE 17 Type Bac — Gestion des notes d'une classe (Fonctions, Tableaux, Enregistrements)

Contexte : Un enseignant du Lycée Bilingue de Yaoundé souhaite automatiser le calcul des moyennes de sa classe de 1ère TI. La classe compte N élèves ($1 \leq N \leq 50$). Pour chaque élève, on connaît : `Matricule` (chaîne), `Nom` (chaîne), et un tableau de 5 notes (réelles) : `Notes[1..5]`.

Travail demandé :

1. Définir le type `Eleve` (enregistrement) adapté à cette description.
2. Écrire une fonction `MoyenneEleve(Notes : Tableau[1..5] de Réels) : Réel` qui calcule et retourne la moyenne des 5 notes d'un élève.
3. Écrire une procédure `SaisirClasse(var Classe : Tableau[1..N] d'Eleve, N : Entier)` qui permet de saisir les informations de tous les élèves (matricule, nom, 5 notes). Utilisez `MoyenneEleve` pour calculer et stocker la moyenne directement dans l'enregistrement si vous ajoutez un champ `Moyenne`, ou bien calculez-la à la volée plus tard.
4. Écrire une fonction `MoyenneClasse(Classe : Tableau[1..N] d'Eleve, N : Entier) : Réel` qui calcule la moyenne générale de la classe (moyenne des moyennes des élèves).
5. Écrire une procédure `AfficherAdmis(Classe : Tableau[1..N] d'Eleve, N : Entier)` qui affiche la liste (`Matricule`, `Nom`, `Moyenne`) des élèves ayant une moyenne ≥ 10 , ainsi que le taux de réussite (pourcentage d'admis).
6. Écrire l'algorithme principal qui déclare les variables, saisit `N`, appelle `SaisirClasse`, puis affiche la moyenne générale de la classe (via `MoyenneClasse`) et la liste des admis (via `AfficherAdmis`).

EXERCICE 18 Type Bac — Gestion de stock d'une boutique informatique (Enregistrements, Tableaux, Procédures/Fonctions)

Contexte : La boutique "TechPlus" à Douala gère un stock de 20 produits maximum. Un produit est défini par : Code (entier), Designation (chaîne), PrixUnitaire (réel), QuantiteStock (entier).

Travail demandé :

1. Déclarer le type `Produit` (enregistrement) et le tableau `Stock[1..20]` de `Produit`. Déclarer une variable `NbProduits` (entier) pour le nombre réel de produits en stock.
2. Écrire une fonction `ValeurStock(P : Produit) : Réel` qui retourne la valeur totale du stock pour ce produit (`PrixUnitaire × QuantiteStock`).
3. Écrire une procédure `AjouterProduit(var Stock : Tableau[1..20] de Produit, var Nb : Entier)` qui demande les infos d'un nouveau produit, vérifie que le code n'existe pas déjà (parcours du tableau), et l'ajoute si `Nb < 20`. Afficher un message d'erreur si stock plein ou code existant.
4. Écrire une procédure `RuptureStock(Stock : Tableau[1..20] de Produit, Nb : Entier)` qui affiche la désignation et le code de tous les produits en rupture de stock (`QuantiteStock = 0`). Si aucun, afficher "Aucune rupture".
5. Écrire une fonction `ValeurTotaleStock(Stock : Tableau[1..20] de Produit, Nb : Entier) : Réel` qui retourne la valeur financière totale du stock de la boutique (somme des `ValeurStock` de chaque produit).
6. Écrire l'algorithme principal proposant un menu simple (1. Ajouter, 2. Voir ruptures, 3. Valeur totale, 4. Quitter) en boucle jusqu'à choix 4, appelant les sous-algorithmes ci-dessus.

EXERCICE 19 Type Bac — Analyse, correction et restructuration d'algorithme

Partie A : Analyse de variables et portée On donne l'algorithme suivant (contenant des erreurs volontaires) :

Variables
X, Y : Entier
Debut
X ← -5
Y ← -10
Afficher "Debut Principal : X=", X, " Y=", Y
Appeler Proc1(X, Y)
Afficher "Après Proc1 : X=", X, " Y=", Y
Z ← Fonction1(X) // Erreur 1 ?
Afficher "Z = ", Z
Fin

```

Procédure Proc1(A : Entier, var B : Entier)
  Var X : Entier // Variable locale nommée X
  Debut
    X <- 20
    A <- A + 5
    B <- B + X
    Afficher "Dans Proc1 : A=", A, " B=", B, " X(local)=", X
  Fin

Fonction Fonction1(N : Entier) : Entier
  Debut
    Retourner N * 2
  Fin

```

1. Identifiez les variables globales, les variables locales à Proc1, les paramètres formels de Proc1 et leur mode de passage (valeur ou référence), les paramètres effectifs de l'appel Appeler Proc1(X, Y).
2. Tracez l'exécution : donnez les valeurs affichées par les trois instructions Afficher.
3. L'instruction Z <- Fonction1(X) (dans le principal) contient une erreur de compilation. Laquelle? Corrigez-la.

Partie B : Correction d'erreurs classiques Corrigez les trois erreurs suivantes dans les fragments de code ci-dessous (recopiez le fragment corrigé uniquement) :

1. Dépassement d'indice :

```

Tableau T[1..10] d'entiers
Pour i de 0 à 10 faire
  T[i] <- i * 2
FinPour

```

2. Appel de procédure dans une affectation :

```

Procédure AfficherDouble(V : Entier)
  Debut
    Afficher V * 2
  Fin

// Dans le principal :
Resultat <- AfficherDouble(5)

```

3. Variable locale utilisée hors de sa portée :

```

Procédure Calculer()
  Var Temp : Entier
  Debut
    Temp <- 100

```

```

    Fin
    // Dans le principal :
    Afficher Temp

```

Partie C : Restructuration (Factorisation) On donne l'algorithme principal suivant, répétitif et non structuré :

```

Debut
    // Traitement Eleve 1
    Saisir Nom1, Note1, Note2, Note3
    Moy1 <- (Note1+Note2+Note3)/3
    Si Moy1 >= 10 Alors Afficher Nom1, " Admis" Sinon Afficher Nom1, " Echech" FinSi

    // Traitement Eleve 2 (copier-coller modifié)
    Saisir Nom2, Note1, Note2, Note3
    Moy2 <- (Note1+Note2+Note3)/3
    Si Moy2 >= 10 Alors Afficher Nom2, " Admis" Sinon Afficher Nom2, " Echech" FinSi

    // Traitement Eleve 3 ...
    // Traitement Eleve 4 ...
Fin

```

1. Proposez une fonction `CalculerMoyenne(n1, n2, n3 : Reel) : Reel`.
2. Proposez une procédure `TraiterEleve()` qui saisit le nom et les 3 notes, appelle `CalculerMoyenne`, et affiche le résultat (Admis/Échec).
3. Réécrivez l'algorithme principal structuré utilisant une boucle `Pour` (pour 4 élèves) appelant `TraiterEleve`.
4. Justifiez en deux phrases pourquoi le choix `Fonction` pour la moyenne et `Procédure` pour le traitement global est pertinent ici.

12 CORRIGÉS DES EXERCICES

13 CORRIGÉS DES EXERCICES — STRUCTURES DE DONNÉES ET SOUS-ALGORITHMES

CORRIGÉ Exercice 1 — QCM : Structures de données et sous-algorithmes

1. Réponse A. En pseudo-code de style MINESEC (celui du programme officiel et d'Algobox), la déclaration d'un tableau dans la zone Variables s'écrit `T : Tableau[1..20]` d'entiers (ou `Tableau[1..20] d'entiers` selon les variantes). La réponse B est une affectation (invalide en déclaration), la C est une syntaxe Visual Basic, et la D mélange deux styles (le mot-clé `Var` n'est pas utilisé pour les déclarations globales dans ce formalisme).
2. Réponse B. L'**enregistrement** (ou structure) regroupe sous un même nom des champs de types potentiellement différents (exemple : `Matricule en chaîne`, `Moyenne en réel`). La réponse A décrit un tableau (même type), la C une boucle `Pour`, la D une variable locale.
3. Réponse A. C'est la définition fondamentale : une **fonction** retourne exactement une valeur via l'instruction `Retourner` et s'utilise dans une expression ; une **procédure** effectue un traitement (éventuellement avec effets de bord) sans retourner de valeur. La réponse D décrit une différence d'usage (syntaxe d'appel) qui découle de A, mais la différence essentielle demandée est le retour de valeur.
4. Réponse B. `a` et `b`, déclarés dans l'en-tête du sous-algorithme, sont les **paramètres formels** : ce sont des « noms de réception » (variables locales au sous-algorithme initialisées par l'appel). Les valeurs/variables transmises à l'appel (`5` et `x`) sont les **paramètres effectifs** (ou arguments).

CORRIGÉ Exercice 2 — Vrai ou Faux ? Justifiez

1. Faux. Si le tableau est déclaré `T : Tableau[1..N]` d'entiers, les indices valides vont de 1 à N : le premier élément est `T[1]`. L'indice de départ vaut 0 uniquement si l'on déclare explicitement `T : Tableau[0..N-1]` d'entiers (convention utilisée en C, JavaScript, Python, mais pas par défaut en pseudo-code MINESEC).
2. Faux. Une variable déclarée dans la zone `Var` d'une fonction ou procédure est une variable **locale** : sa portée est limitée au corps de ce sous-algorithme ; elle est créée

à l'appel et détruite au retour. Elle est donc invisible (inaccessible) depuis le programme principal ou un autre sous-algorithme.

3. Vrai. En passage **par valeur** (défaut si `var` absent), le sous-algorithme travaille sur une copie de l'argument : toute modification du paramètre formel n'affecte pas la variable du programme appelant. (Pour modifier l'argument original, il faut un passage par référence, signalé par le mot-clé `var` devant le paramètre formel.)
4. Faux. Une procédure ne retourne pas de valeur ; elle ne peut donc pas apparaître dans une expression arithmétique ni à droite d'une affectation. `X <- 5 + MaProc(3)` provoque une erreur de syntaxe/sémantique. Seule une fonction peut être utilisée ainsi : `X <- 5 + MaFonction(3)`.
5. Vrai (notation standard). En pseudo-code MINESEC, l'accès à un champ d'enregistrement se fait par le point : `P.Prix`. La notation `P->Prix` est celle du langage C pour un pointeur vers structure ; elle n'est pas attendue au Probatoire.

CORRIGÉ Exercice 3 — Définitions et vocabulaire

1. Enregistrement (structure) : structure de données composite qui regroupe sous un même nom plusieurs champs de types éventuellement différents, décrivant une même entité. Exemple :

Type Eleve = Enregistrement
Matricule : Chaîne
Nom : Chaîne
Moyenne : Reel
FinEnregistrement

2. Variable locale / globale (portée) : une variable **globale** est déclarée dans la zone **Variables** du programme principal (avant **Debut**) ; elle est visible et utilisable partout, y compris dans les sous-algorithmes. Une variable **locale** est déclarée dans la zone **Var** d'une fonction ou procédure ; elle n'existe que pendant l'exécution de ce sous-algorithme et est invisible à l'extérieur.
3. Paramètre formel / effectif : le **paramètre formel** est la variable déclarée dans l'entête du sous-algorithme (ex. `Fonction Carre(n : Entier) : n` est formel). Le **paramètre effectif** (argument) est la valeur ou la variable transmise lors de l'appel (ex. `Carre(7) : 7` est l'argument ; `Carre(x) : x` est l'argument). À l'appel, chaque paramètre effectif est associé au paramètre formel de même position (ordre strict).
4. Effet de bord : modification, par une procédure ou une fonction, d'une variable globale ou d'un paramètre passé par référence (`var`), c'est-à-dire une modification visible en dehors du sous-algorithme. Les effets de bord rendent les algorithmes difficiles à tracer, tester et déboguer ; on les limite en privilégiant les variables locales, les

passages par valeur et les fonctions pures (sans effet de bord).

CORRIGÉ Exercice 4 — Lecture et exécution manuelle (traçage)

1. Valeur affichée : $c = 13$.

Traçage détaillé : le tableau T contient $T[1]=4$, $T[2]=7$, $T[3]=2$. L'appel $\text{SommePremiers}(T, 3)$ exécute la boucle pour i de 1 à 3 :

- $i=1 : s \leftarrow 0 + 4 = 4$
- $i=2 : s \leftarrow 4 + 7 = 11$
- $i=3 : s \leftarrow 11 + 2 = 13$

La fonction retourne 13, donc $c \leftarrow 13$. Vérification : $4 + 7 + 2 = 13$. (Vérifié)

2. Appel $\text{SommePremiers}(T, 6)$: la boucle atteint $i = 6$ et tente d'accéder à $T[6]$, alors que le tableau est déclaré $T[1..5]$. C'est un **dépassement d'indice** (accès hors limites) : erreur d'exécution (ou comportement indéfini selon l'interpréteur). Il faut toujours vérifier que le paramètre n ne dépasse pas la taille déclarée du tableau.
3. i et s sont des variables locales : elles sont déclarées dans la zone `Var` de la fonction SommePremiers . Elles sont créées à chaque appel, invisibles depuis le programme principal, et détruites au `Retourner`. C'est ce qui permet d'appeler la fonction plusieurs fois (ou récursivement) sans interférence entre les appels.

CORRIGÉ Exercice 5 — Manipulation de tableau à une dimension

Algorithme AnalyseNotes

Variables

$N, i, \text{NbAdmis}$: Entier

Notes : Tableau[1..50] de Reels

Max : Reel

PosMax : Entier

Procédure SaisieTableau(var T : Tableau[1..50] de Reels, n : Entier)

Var

i : Entier

Debut

Pour i de 1 à n faire

 Ecrire "Note numero ", i , " : "

 Lire $T[i]$

FinPour

Fin

```
Fonction Maximum(T : Tableau[1..50] de Reels, n : Entier) : Reel
```

```
Var
```

```
  i : Entier
```

```
  m : Reel
```

```
Debut
```

```
  m <- T[1]
```

```
  Pour i de 2 a n faire
```

```
    Si T[i] > m Alors
```

```
      m <- T[i]
```

```
    FinSi
```

```
  FinPour
```

```
  Retourner m
```

```
Fin
```

```
Debut
```

```
  Repeter
```

```
    Ecrire "Taille du tableau (1 a 50) : "
```

```
    Lire N
```

```
  Jusqu'a (N >= 1) ET (N <= 50)
```

```
  SaisieTableau(Notes, N)
```

```
  Max <- Maximum(Notes, N)
```

```
  // Recherche de la PREMIERE occurrence du maximum
```

```
  PosMax <- 1
```

```
  i <- 2
```

```
  TantQue (i <= N) ET (Notes[i] <> Max) faire
```

```
    i <- i + 1
```

```
  FinTantQue
```

```
  Si i <= N Alors
```

```
    PosMax <- i
```

```
  FinSi
```

```
  NbAdmis <- 0
```

```
  Pour i de 1 a N faire
```

```
    Si Notes[i] >= 10 Alors
```

```
      NbAdmis <- NbAdmis + 1
```

```
    FinSi
```

```
  FinPour
```

```
  Ecrire "Plus grande note : ", Max, " a la position ", PosMax
```

```
  Ecrire "Nombre de notes >= 10 : ", NbAdmis
```

```
Fin
```

Points de vigilance :

- Le maximum s'initialise à T[1] (jamais à 0 : si toutes les notes étaient négatives, le résultat serait faux).
- La recherche de la position doit comparer à Max après le calcul du maximum.

- Pour obtenir la première occurrence, on utilise une boucle `TantQue` avec arrêt dès que `Notes[i] = Max` (ou une boucle `Pour` avec une instruction de sortie anticipée si le langage le permet ; en pseudo-code standard, `TantQue` est plus clair).
- Les accumulateurs (`NbAdmis`, `s` dans `Maximum`) doivent être initialisés à 0 avant les boucles.
- Le paramètre `T de SaisieTableau` est passé par `var` (référence) car la procédure le modifie ; celui de `Maximum` est passé par valeur (pas de `var`) car la fonction ne fait que le lire.

CORRIGÉ Exercice 6 — Enregistrement et tableau d'enregistrements

1. Déclarations :

```
Type Eleve = Enregistrement
  Matricule : Chaîne
  Nom : Chaîne
  Moyenne : Reel
FinEnregistrement

Variables
  Classe : Tableau[1..40] d'Eleve
  N, i : Entier
  Meilleur : Eleve
```

2. Procédure de saisie (passage par référence obligatoire via `var`, sinon la modification ne porterait que sur une copie locale) :

```
Procédure SaisirEleve(var E : Eleve)
  Debut
    Ecrire "Matricule : " ; Lire E.Matricule
    Ecrire "Nom : " ; Lire E.Nom
    Ecrire "Moyenne : " ; Lire E.Moyenne
  Fin
```

3. Fonction retournant le meilleur élève (enregistrement complet) :

```
Fonction MeilleurEleve(C : Tableau[1..40] d'Eleve, n : Entier) : Eleve
  Var
    i, iMax : Entier
  Debut
    iMax ← -1
    Pour i de 2 à n faire
      Si C[i].Moyenne > C[iMax].Moyenne Alors
        iMax ← i
      FinSi
    FinPour
```

```
Retourner C[iMax]
```

```
Fin
```

4. Algorithme principal :

```
Algorithme GestionClasse
```

```
Variables
```

```
Classe : Tableau[1..40] d'Eleve
```

```
N, i : Entier
```

```
Meilleur : Eleve
```

```
Debut
```

```
Repeter
```

```
  Ecrire "Nombre d'eleves (1 a 40) : "
```

```
  Lire N
```

```
Jusqua (N >= 1) ET (N <= 40)
```

```
  Pour i de 1 a N faire
```

```
    SaisirEleve(Classe[i])
```

```
  FinPour
```

```
  Meilleur <- MeilleurEleve(Classe, N)
```

```
  Ecrire "Meilleur eleve : ", Meilleur.Nom,  
    " avec ", Meilleur.Moyenne
```

```
Fin
```

Points de vigilance :

- Le mot-clé **var** dans **SaisirEleve** est indispensable (passage par référence) : sans lui, E serait une copie locale et **Classe[i]** ne serait pas modifié.
- On accède aux champs par le point : **C[i].Moyenne**, **Meilleur.Nom**.
- La fonction retourne ici un enregistrement complet, ce qui est licite en pseudo-code (mais attention : en C, on retournerait un pointeur ou une structure par valeur avec précautions).
- Contrôler la saisie de **N** ($1 \leq N \leq 40$) pour éviter les débordements.

CORRIGÉ

Exercice 7 — Fonction récursive vs itérative : Factorielle

1. Version itérative :

```
Fonction FactIter(N : Entier) : Entier
```

```
Var
```

```
  i, f : Entier
```

```
Debut
```

```
  Si N < 0 Alors
```

```
    Retourner -1 // factorielle non définie pour les négatifs
```

```

FinSi
f <- 1
Pour i de 1 a N faire
    f <- f * i
FinPour
Retourner f
Fin

```

Vérification : $N = 4 \Rightarrow f = 1 \times 1 \times 2 \times 3 \times 4 = 24$. Pour $N = 0$, la boucle ne s'exécute pas et $f = 1 = 0!$. (Vérifié)

2. Version récursive :

```

Fonction FactRec(N : Entier) : Entier
Debut
    Si N < 0 Alors
        Retourner -1
    FinSi
    Si N = 0 Alors
        Retourner 1 // cas de base (arrêt de la recursion)
    Sinon
        Retourner N * FactRec(N - 1) // appel récursif
    FinSi
Fin

```

Traçage : $\text{FactRec}(3) = 3 \times \text{FactRec}(2) = 3 \times 2 \times \text{FactRec}(1) = 3 \times 2 \times 1 \times \text{FactRec}(0) = 3 \times 2 \times 1 \times 1 = 6$. (Vérifié)

3. Algorithme principal de test :

```

Algorithme TestFactorielle
Variables
    N : Entier
Debut
    Ecrire "Entrez N : "
    Lire N
    Ecrire "Itératif : ", FactIter(N)
    Ecrire "Récursif : ", FactRec(N)
Fin

```

4. Avantage de la version itérative : chaque appel récursif empile un contexte (paramètres, variables locales, adresse de retour) sur la **pile d'exécution** ; pour N grand, cela consomme beaucoup de mémoire et peut provoquer un débordement de pile (Stack Overflow). La version itérative n'utilise qu'un nombre constant de variables (i et f), donc une mémoire constante $O(1)$.

CORRIGÉ Exercice 8 — Portée des variables et passage de paramètres

1. Algorithme complété :

```

Algorithme TestPortee
Variables
  X, Y, Z : Entier      // variables globales

Debut
  X ← 5
  Y ← 10
  Modifier(X, Y)        // Appel 1
  Ecrire "Après Appel 1 : X=", X, " Y=", Y
  Z ← Calculer(X, Y)    // Appel 2
  Ecrire "Z = ", Z

Fin

Procédure Modifier(var A : Entier, B : Entier)
Debut
  A ← 20
  B ← 30
Fin

Fonction Calculer(P, Q : Entier) : Entier
Var
  Locale : Entier
Debut
  Locale ← P + Q
  Retourner Locale
Fin

```

2. Après `Modifier(X, Y)` : `X = 20` et `Y = 10` (inchangé).

Justification :

- A est passé **par référence** (`var A`) : A et X désignent la même case mémoire, donc `A ← 20` modifie X.
- B est passé **par valeur** (pas de `var`) : B est une copie de Y ; l'affectation `B ← 30` ne modifie que la copie locale, Y reste à 10.

Ensuite `Z ← Calculer(20, 10)`. Dans `Calculer`, `P=20`, `Q=10` (passage par valeur), `Locale = 30`, retour 30. Donc `Z = 30`.

3. Inventaire :

- Variables globales : X, Y, Z.
- Variables locales à `Modifier` : aucune (elle n'a que ses paramètres A et B).
- Variables locales à `Calculer` : `Locale`.
- Paramètres formels de `Modifier` : A (par référence) et B (par valeur).

- Paramètres effectifs de l'appel 1 : X et Y.

CORRIGÉ Exercice 9 — Type Bac : Gestion des notes d'une classe

Barème indicatif (20 pts) : type Eleve : 3 pts; MoyenneEleve : 3 pts; SaisirClasse : 4 pts; MoyenneClasse : 3 pts; AfficherAdmis (avec taux) : 4 pts; algorithme principal : 3 pts.

1. Type enregistrement :

```
Type Eleve = Enregistrement
  Matricule : Chaîne
  Nom : Chaîne
  Notes : Tableau[1..5] de Reels
  Moyenne : Reel
FinEnregistrement
```

2. Fonction de calcul de la moyenne d'un élève :

```
Fonction MoyenneEleve(Notes : Tableau[1..5] de Reels) : Reel
Var
  i : Entier
  s : Reel
Debut
  s <- 0
  Pour i de 1 a 5 faire
    s <- s + Notes[i]
  FinPour
  Retourner s / 5
Fin
```

3. Procédure de saisie de la classe :

```
Procédure SaisirClasse(var Classe : Tableau[1..50] d'Eleve, N : Entier)
Var
  i, j : Entier
Debut
  Pour i de 1 a N faire
    Ecrire "Eleve ", i, " : Matricule ? "
    Lire Classe[i].Matricule
    Ecrire "Nom ? "
    Lire Classe[i].Nom
    Pour j de 1 a 5 faire
      Ecrire "Note ", j, " : "
      Lire Classe[i].Notes[j]
    FinPour
    Classe[i].Moyenne <- MoyenneEleve(Classe[i].Notes)
  FinPour
```

Fin

4. Moyenne générale de la classe :

Fonction MoyenneClasse(Classe : Tableau[1..50] d'Eleve, N : Entier) : Reel

Var

i : Entier

s : Reel

Debut

s ← 0

Pour i de 1 a N faire

s ← s + Classe[i].Moyenne

FinPour

Retourner s / N

Fin

5. Affichage des admis et taux de réussite :

Procedure AfficherAdmis(Classe : Tableau[1..50] d'Eleve, N : Entier)

Var

i, NbAdmis : Entier

Debut

NbAdmis ← 0

Ecrire "Liste des admis :"

Pour i de 1 a N faire

Si Classe[i].Moyenne ≥ 10 Alors

Ecrire Classe[i].Matricule, " ",

Classe[i].Nom, " ",

Classe[i].Moyenne

NbAdmis ← NbAdmis + 1

FinSi

FinPour

// Attention : division réelle pour le pourcentage

Ecrire "Taux de réussite : ",

(NbAdmis * 100.0) / N, " %"

Fin

6. Algorithme principal :

Algorithme GestionNotes

Variables

Classe : Tableau[1..50] d'Eleve

N : Entier

Debut

Repeter

Ecrire "Nombre d'eleves (1 a 50) : "

Lire N

Jusqua (N ≥ 1) ET (N ≤ 50)

```

SaisirClasse(Classe, N)
Ecrire "Moyenne generale : ", MoyenneClasse(Classe, N)
AfficherAdmis(Classe, N)
Fin

```

Points de vigilance :

- Justifier le **var** devant **Classe** dans **SaisirClasse** (le tableau est modifié : passage par référence obligatoire).
- Le taux de réussite exige la formule $\frac{\text{NbAdmis} \times 100}{N}$; en pseudo-code, écrire `100.0` ou `Reel(NbAdmis)` pour forcer une division réelle et éviter une division entière tronquée.
- Contrôler la saisie de N ($1 \leq N \leq 50$).
- Les accumulateurs (`s`, `NbAdmis`) doivent être initialisés à 0 avant les boucles.

CORRIGÉ Exercice 10 — Type Bac : Gestion de stock de la boutique TechPlus

Barème indicatif (20 pts) : déclarations : 3 pts ; ValeurStock : 2 pts ; AjouterProduit (avec vérifications) : 5 pts ; RuptureStock : 3 pts ; ValeurTotaleStock : 3 pts ; menu principal : 4 pts.

1. Déclarations :

```

Type Produit = Enregistrement
    Code : Entier
    Designation : Chaine
    PrixUnitaire : Reel
    QuantiteStock : Entier
FinEnregistrement

Variables
    Stock : Tableau[1..20] de Produit
    NbProduits : Entier

```

2. Valeur du stock d'un produit :

```

Fonction ValeurStock(P : Produit) : Reel
    Debut
        Retourner P.PrixUnitaire * P.QuantiteStock
    Fin

```

3. Ajout d'un produit avec contrôles (unicité du code, place dispo) :

```

Procédure AjouterProduit(var Stock : Tableau[1..20] de Produit,
                        var Nb : Entier)
Var

```

```

i : Entier
Nouveau : Produit
Existe : Booleen
Debut
Si Nb >= 20 Alors
    Ecrire "Erreur : stock plein (20 produits max)"
Sinon
    Ecrire "Code : " ; Lire Nouveau.Code
    Existe <- Faux
    Pour i de 1 a Nb faire
        Si Stock[i].Code = Nouveau.Code Alors
            Existe <- Vrai
        FinSi
    FinPour
    Si Existe Alors
        Ecrire "Erreur : code deja existant"
    Sinon
        Ecrire "Designation : " ; Lire Nouveau.Designation
        Ecrire "Prix unitaire : " ; Lire Nouveau.PrixUnitaire
        Ecrire "Quantite : " ; Lire Nouveau.QuantiteStock
        Nb <- Nb + 1
        Stock[Nb] <- Nouveau
        Ecrire "Produit ajoute avec succes"
    FinSi
FinSi
Fin

```

4. Produits en rupture (quantité = 0) :

```

ProcEDURE RuptureStock(Stock : Tableau[1..20] de Produit, Nb : Entier)
Var
    i, cpt : Entier
Debut
    cpt <- 0
    Pour i de 1 a Nb faire
        Si Stock[i].QuantiteStock = 0 Alors
            Ecrire Stock[i].Code, " - ", Stock[i].Designation
            cpt <- cpt + 1
        FinSi
    FinPour
    Si cpt = 0 Alors
        Ecrire "Aucune rupture"
    FinSi
Fin

```

5. Valeur financière totale du stock :

```

Fonction ValeurTotaleStock(Stock : Tableau[1..20] de Produit,
Nb : Entier) : Reel

```

```
Var
  i : Entier
  total : Reel
Debut
  total <- 0
  Pour i de 1 a Nb faire
    total <- total + ValeurStock(Stock[i])
  FinPour
  Retourner total
Fin
```

6. Algorithme principal avec menu :

```
Algorithme GestionStock
Variables
  Stock : Tableau[1..20] de Produit
  NbProduits, Choix : Entier
Debut
  NbProduits <- 0
  Repeter
    Ecrire "1. Ajouter 2. Ruptures 3. Valeur totale 4. Quitter"
    Lire Choix
    Selon Choix faire
      1 : AjouterProduit(Stock, NbProduits)
      2 : RuptureStock(Stock, NbProduits)
      3 : Ecrire "Valeur totale : ",
          ValeurTotaleStock(Stock, NbProduits), " FCFA"
      4 : Ecrire "Au revoir"
      Sinon : Ecrire "Choix invalide"
    FinSelon
  Jusqu'à Choix = 4
Fin
```

Points de vigilance :

- **NbProduits doit être passé par référence (var) dans AjouterProduit car il est modifié (incrémenté).**
- **L'initialisation NbProduits <- 0 avant la boucle Repeter est cruciale.**
- **La recherche de code existant ne parcourt que les Nb premières cases valides (indices 1 à Nb), pas les 20 cases du tableau.**
- **Penser au cas « aucune rupture » via un compteur cpt pour afficher un message clair.**
- **La fonction ValeurTotaleStock réutilise ValeurStock : factorisation correcte.**

CORRIGÉ Exercice 11 — Type Bac : Analyse, correction et restructuration

Barème indicatif (20 pts) : Partie A : 7 pts ; Partie B : 6 pts (2 pts par correction) ; Partie C : 7 pts.

Partie A — Analyse de variables et portée**1. Inventaire :**

- Variables globales : `X` et `Y`.
- Variables locales à `Proc1` : `X` (locale, qui masque la globale `X` à l'intérieur de `Proc1`).
- Paramètres formels de `Proc1` : `A` (passage par valeur) et `B` (passage par référence, mot-clé `var`).
- Paramètres effectifs de l'appel : `X` (associé à `A`) et `Y` (associé à `B`).

2. Traçage pas à pas :

- Début principal : `X=5, Y=10`. Affichage 1 : `Debut Principal : X=5 Y=10`.
- Appel `Proc1(X, Y)` :
 - `A (valeur)` reçoit copie de `X (5)`. `B (référence)` alias de `Y`.
 - `X(local) <- 20` (masque le global).
 - `A <- A + 5` $\Rightarrow$ `A = 10` (local, n'affecte pas global `X`).
 - `B <- B + 20` $\Rightarrow$ `B = 30` $\Rightarrow$ `Y = 30` (référence).
 - Affichage 2 : Dans `Proc1` : `A=10 B=30 X(local)=20`.
- Retour au principal : `X = 5` (inchangé, passage par valeur), `Y = 30`. Affichage 3 : Après `Proc1` : `X=5 Y=30`.

3. Erreur : la variable `Z` n'est pas déclarée dans la zone des variables globales. Correction : ajouter `Z : Entier` dans les déclarations. Après correction, `Z <- Fonction1(5)` = 10, et l'affichage donne `Z = 10`.

Partie B — Correction d'erreurs classiques

1. Erreur : Dépassement d'indice. Le tableau est déclaré `T[1..10]`, donc les indices valides sont 1 à 10. La boucle Pour `i` de 0 à 10 accède à `T[0]` (invalide) et `T[10]` (valide). Fragment corrigé :

```
Tableau T[1..10] d'entiers
Pour i de 1 a 10 faire
  T[i] <- i * 2
FinPour
```

2. Erreur : Une procédure ne retourne rien (pas de `Retourner`), on ne peut pas l'affecter à une variable. Fragment corrigé (on appelle la procédure pour son effet d'affichage) :

```
// Dans le principal :
AfficherDouble(5)
```

(Alternative : transformer AfficherDouble en fonction Double(V : Entier) : Entier qui retourne V * 2, puis écrire Resultat <- Double(5).)

- Erreur : Temp est locale à Calculer : elle n'existe plus dans le programme principal. Correction : soit déclarer Temp en variable globale (mauvaise pratique), soit (mieux) utiliser une fonction qui retourne la valeur :

```

Fonction Calculer() : Entier
Var
    Temp : Entier
Debut
    Temp <- 100
    Retourner Temp
Fin
// Dans le principal :
Ecrire Calculer()

```

Partie C — Restructuration (factorisation)

- Fonction de calcul de moyenne (réutilisable, sans effet de bord) :

```

Fonction CalculerMoyenne(n1, n2, n3 : Reel) : Reel
Debut
    Retourner (n1 + n2 + n3) / 3
Fin

```

- Procédure de traitement d'un élève (suite d'actions, saisie/affichage) :

```

Procédure TraiterEleve()
Var
    Nom : Chaîne
    n1, n2, n3, Moy : Reel
Debut
    Ecrire "Nom : " ; Lire Nom
    Ecrire "3 notes : " ; Lire n1, n2, n3
    Moy <- CalculerMoyenne(n1, n2, n3)
    Si Moy >= 10 Alors
        Ecrire Nom, " Admis (", Moy, ")"
    Sinon
        Ecrire Nom, " Echec (", Moy, ")"
    FinSi
Fin

```

- Algorithme principal structuré (boucle + appel) :

```

Algorithme BilanClasse
Variables
    i : Entier
Debut
    Pour i de 1 à 4 faire

```

```

|   Ecrire " --- Eleve ", i, " ---"
|   TraiterEleve()
|   FinPour
|   Fin

```

4. Justification du choix Fonction vs Procédure :

- CalculerMoyenne : produit une unique valeur réutilisable dans une expression (Moy <- ...); pas d'affichage, pas de modification d'état global. C'est le rôle exact d'une **fonction** (retour de valeur, transparence référentielle).
- TraiterEleve : effectue une suite d'actions (saisies utilisateur, affichages résultats) qui ne produit pas de valeur à réutiliser dans une expression. C'est le rôle d'une **procédure** (effets de bord sur l'entrée/sortie standard).

Cette factorisation élimine le copier-coller, réduit les risques d'erreur et rend l'algorithme facilement extensible (il suffit de changer la borne de la boucle pour traiter 30 élèves).

Points de vigilance :

- Dans la Partie A, exiger la distinction explicite entre le `x` local (qui masque le global) et le `x` global ; c'est un piège classique du Probatoire.
- Dans la Partie B, la justification du mode de passage (valeur/référence) et de la portée (locale/global) vaut autant de points que la correction syntaxique.
- Dans la Partie C, toute solution sans boucle Pour ni sous-algorithmes (Fonction + Procédure) ne répond pas à la consigne de « restructuration » et perd la majorité des points.

14 FICHE BILAN

Carte mentale de la leçon : les six idées maîtresses à retenir.

FICHE BILAN L'essentiel à connaître pour l'examen

Définitions clés

- **Structure de données** : manière organisée de stocker et de manipuler un ensemble

de données (tableau, enregistrement, chaîne, fichier...).

- **Tableau** : structure regroupant des éléments de même type, accessibles par un **indice** (de 1 à N en algorithmique).
- **Enregistrement** : structure regroupant des **champs** de types éventuellement différents ; accès par la notation `variable.champ`.
- **Sous-algorithme** : bloc d'instructions nommé, déclaré hors de l'algorithme principal, et **appelé** depuis celui-ci.
- **Fonction** : sous-algorithme qui calcule et retourne une seule valeur (Retourner).
- **Procédure** : sous-algorithme qui réalise une action (affichage, modification) sans retourner de valeur.
- **Variable globale** : déclarée dans l'algorithme principal, visible partout. **Variable locale** : déclarée dans un sous-algorithme, visible uniquement dans celui-ci.
- **Paramètre formel** : nom utilisé dans la déclaration du sous-algorithme. **Paramètre effectif** (argument) : valeur réellement transmise lors de l'appel.

Syntaxes à connaître par cœur

Notion	Écriture algorithmique
Déclarer un tableau	Variable T : Tableau[1..10] de Réel
Accéder à un élément	T[i] ← 12,5 (indice entre 1 et N)
Déclarer un enregistrement	Type Eleve = Enregistrement; champs ; Fin
Accéder à un champ	e.nom ← "Manga"
Déclarer une fonction	Fonction carre(x : Réel) : Réel ... Retourner x*x
Appeler une fonction	y ← carre(4)
Déclarer une procédure	Procédure afficher(msg : Chaîne) ... Fin
Appeler une procédure	afficher("Bonjour")

Méthodes express

1. Parcourir un tableau : **boucle** Pour i de 1 à N ; lire ou modifier T[i] ; ne jamais dépasser N.
2. Choisir fonction ou procédure : le sous-algorithme doit-il produire une valeur ? Oui ⇒ fonction ; non ⇒ procédure.
3. Identifier les paramètres : les données nécessaires au calcul deviennent des paramètres ; le résultat est retourné (fonction) ou affiché (procédure).
4. Structurer un algorithme : déclarer les types et sous-algorithmes d'abord, puis écrire un algorithme principal court qui les appelle.

À RETENIR

Checklist avant le Bac

- ☐ Je sais énumérer au moins trois structures de données (tableau, enregistrement, chaîne...).
- ☐ Je sais déclarer un tableau à une dimension avec son nom, sa taille et son type.
- ☐ Je sais initialiser, parcourir, afficher et modifier les éléments d'un tableau avec une boucle Pour.
- ☐ Je sais déclarer un enregistrement et accéder à ses champs avec la notation pointée.
- ☐ Je sais expliquer le rôle d'un sous-algorithme : découper, clarifier, réutiliser.
- ☐ Je sais distinguer fonction (retourne une valeur) et procédure (effectue une action).
- ☐ Je sais écrire l'en-tête correct d'une fonction avec son type de retour.
- ☐ Je sais appeler une fonction ou une procédure depuis l'algorithme principal.
- ☐ Je sais différencier variable locale et variable globale, et justifier leur portée.
- ☐ Je sais distinguer paramètres formels (déclaration) et paramètres effectifs (appel).
- ☐ Je vérifie toujours que le nombre et le type des arguments correspondent aux paramètres.
- ☐ Je ne dépasse jamais les bornes d'un tableau (indice entre 1 et N).



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+